INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

LeGrandHotel Semana11

Funcionalidades, pruebas y diseño móvil

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Descripción del proyecto 3
1.1 Objetivos 3
1.2 Módulos 3
1.3 Tecnologías 3
1.4 Cómo se conectan las partes 4
1.5 Diagrama de la base de datos 5
2. Endpoints definidos 8
2.1 Formato único de respuesta 9
3. Registro y login diferenciados 9
3.1 Fragmentos de código 10
3.2 Pantallas de registro 11
4. Qué puede hacer cada rol 11
4.1 Recepcionista 12
4.2 Almacenero 21
4.3 Administrador 24
4.4 Programador 42
4.5 Cliente 47
5. Pantallas de error 49
6. Registro de incidentes 51
7. Pruebas de rendimiento y estrés con k6 52
7.1 Prueba de humo 53
7.2 Límite de intentos de login 54
7.3 Escalones de clientes (5, 10, 25 y 50 a la vez) 55
7.4 Reporte HTML de k6 (escalón de 50 clientes) 58
7.5 Conclusión de las pruebas 59
8. Diseño de la aplicación Android e iOS 59
8.1 Tecnología 62
8.2 Seguridad y cambios en la API 62
8.3 Plan de construcción 62
9. Conclusiones 63

1. Descripción del proyecto

LeGrandHotel es un sistema web para gestionar un hotel. Reúne en un solo lugar el trabajo diario del personal (reservas, check-in y check-out, habitaciones, huéspedes, cochera e inventario) y la reserva en línea de los huéspedes, que eligen el tipo de habitación, ven lo que incluye y el total, y el hotel les asigna una habitación libre.

El sistema tiene dos partes: una API REST que guarda los datos y aplica todas las reglas del negocio, y una página web (aplicación de una sola página) que la consume. La misma API servirá también a la aplicación móvil para Android e iOS que se diseña en la sección 8.

1.1 Objetivos

- Evitar errores de operación: que una habitación no se reserve dos veces en las mismas fechas, que no haya dos huéspedes en la misma habitación y que el inventario no quede en negativo.
- Que cada persona vea solo lo que le corresponde según su rol, y que la API lo haga cumplir aunque alguien intente entrar escribiendo la dirección a mano.
- Dejar rastro: cada cambio queda en la auditoría con quién lo hizo y cuándo, y cada error tiene un código de incidente para encontrarlo en el registro.
- Medir el rendimiento con pruebas de carga antes de publicar el sistema.

1.2 Módulos

Módulo	Qué cubre
Reservas y estadías	Reservas con cálculo del total en el servidor, check-in (crea la estadía y ocupa la habitación) y check-out (la habitación pasa a limpieza)
Habitaciones y tipos	Habitaciones por piso y estado; tipos con precio por noche, capacidad y lo que incluyen
Huéspedes	Fichas con documento, teléfono y nacionalidad; se vinculan con la cuenta web si el huésped tiene una
Cochera	Espacios, entrada y salida de vehículos, con o sin estadía asociada
Inventario	Insumos con stock y mínimo; entradas y salidas con motivo
Personal y cuentas	Fichas del personal, registro de cuentas y cambio de roles
Reportes y auditoría	Ocupación, ingresos de los últimos 6 meses y quién cambió qué
Portal del cliente	Entrada con Google, datos del huésped, reserva en línea y sus reservas
Vista del programador	Salud de la API, latencias, peticiones, errores y rutas, sin entrar al servidor

1.3 Tecnologías

Parte	Tecnología	Para qué se usa
API (backend)	PHP 8.2 y Laravel 12	Rutas REST, validación, reglas del negocio y acceso a datos
Autenticación	Laravel Sanctum	Tokens Bearer con vencimiento (60 minutos) para la web y la app móvil
Inicio con Google	Laravel Socialite (OAuth 2.0)	Cuentas de clientes sin contraseña
Base de datos	MySQL 8.4	Datos del hotel con claves foráneas, índices y transacciones

Parte	Tecnología	Para qué se usa
Página web (frontend)	React 18, TypeScript y Vite	Interfaz de una sola página, rápida y con tipos
Datos en la web	TanStack Query y Axios	Caché de consultas, paginación y manejo único de errores 401/403/429/5xx
Navegación	React Router	Rutas protegidas por rol y pantallas de error
Pruebas de la API	Pest (PHPUnit)	121 pruebas automáticas de reglas, permisos y errores
Pruebas de la web	Vitest y Testing Library	25 pruebas de rutas, formularios y permisos
Calidad de código	PHPStan nivel 5, Laravel Pint, ESLint y TypeScript estricto	Detectar errores antes de ejecutar y mantener un estilo único
Pruebas de carga	k6 (Grafana)	Medir tiempos de respuesta y errores con muchos usuarios a la vez
Entorno local	Docker Compose	La misma base MySQL para todo el equipo
App móvil (diseño)	React Native con Expo y TypeScript	Una sola base de código para Android e iOS
Control de versiones	Git y GitHub	Historial y trabajo en equipo

Nota: el servidor donde se publicará el sistema todavía no está definido. Todo lo de este informe se ejecutó en el entorno local del equipo.

1.4 Cómo se conectan las partes

La página web y la app móvil no guardan datos: le piden todo a la API en formato JSON con el token de la sesión. La API valida cada petición (FormRequest), revisa el rol (middleware role:) y las reglas por registro (policies), y responde siempre con el mismo formato, sea éxito o error. La base de datos MySQL tiene 16 tablas del sistema relacionadas por 13 claves foráneas; el diagrama de la sección 1.5 se generó leyendo el esquema real de la base.

1.5 Diagrama de la base de datos

El diagrama se divide en tres partes para que se lea bien. Cuando una tabla se relaciona con otra de una parte distinta, esa tabla aparece en un recuadro punteado solo con su id. Cada línea muestra la columna de la clave foránea y qué hace MySQL al borrar el registro padre (CASCADE, SET NULL o RESTRICT).

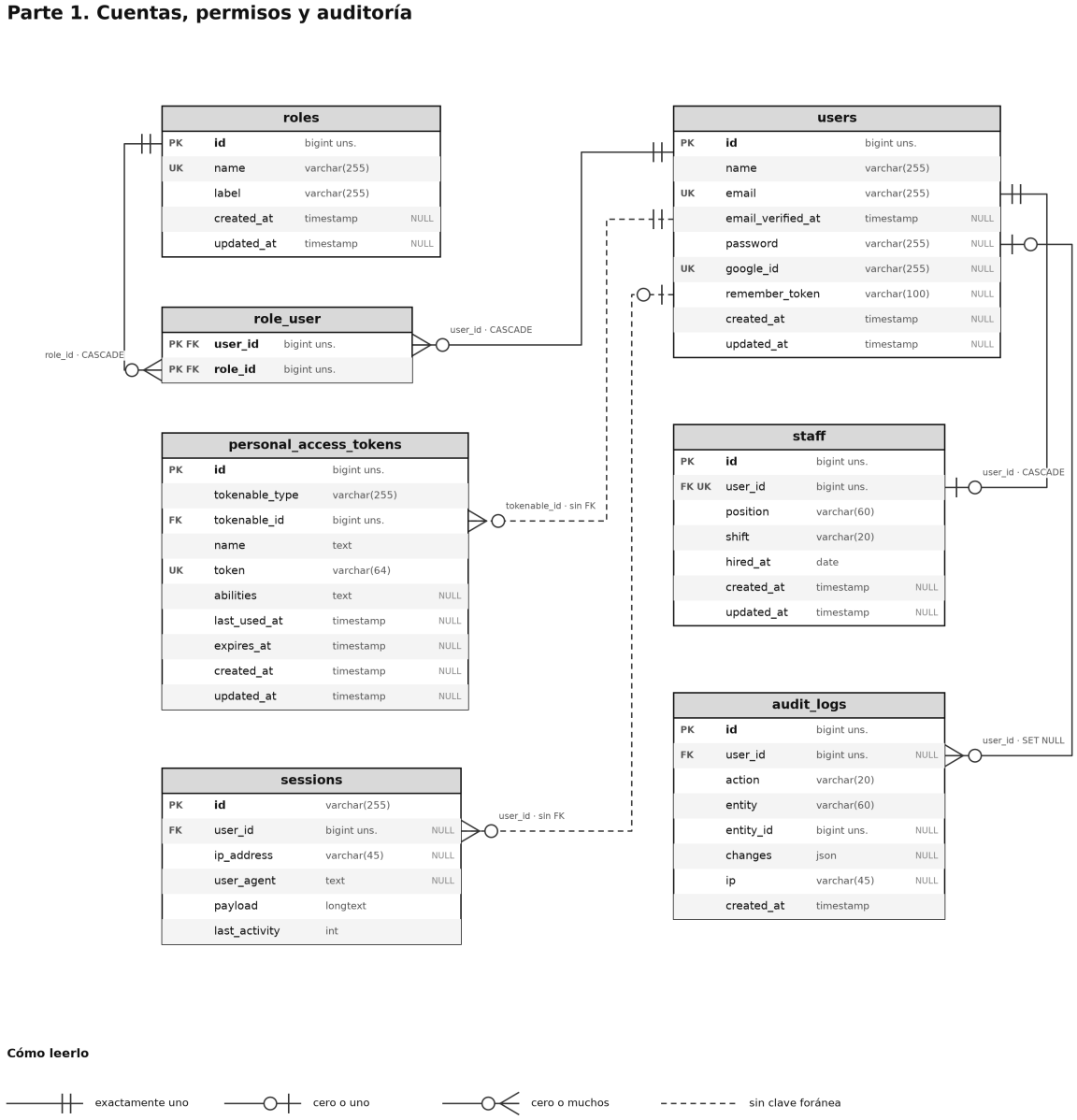


Figura 1. Diagrama de la base de datos, parte 1: cuentas, permisos y auditoría.

Parte 2. Huéspedes, reservas y habitaciones

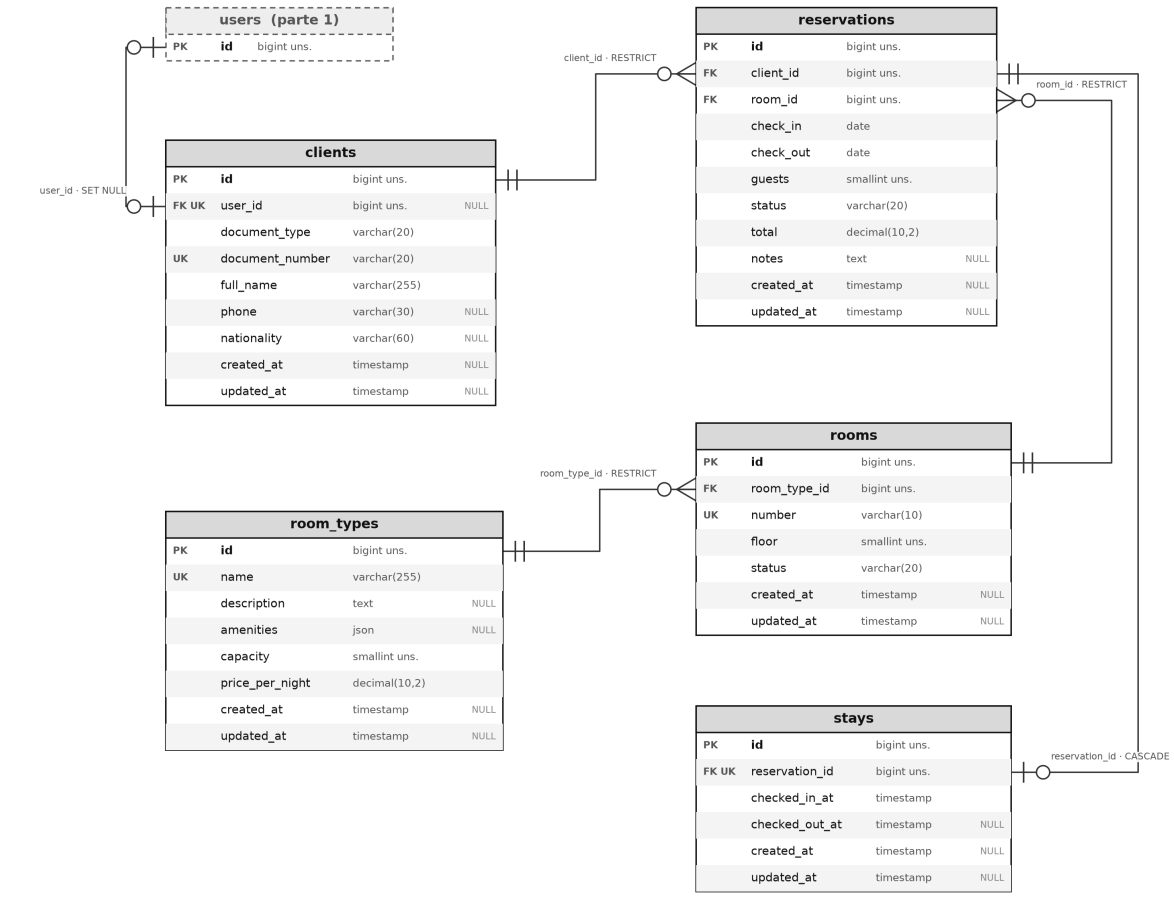


Figura 2. Diagrama de la base de datos, parte 2: huéspedes, reservas y habitaciones.

Parte 3. Cochera e inventario

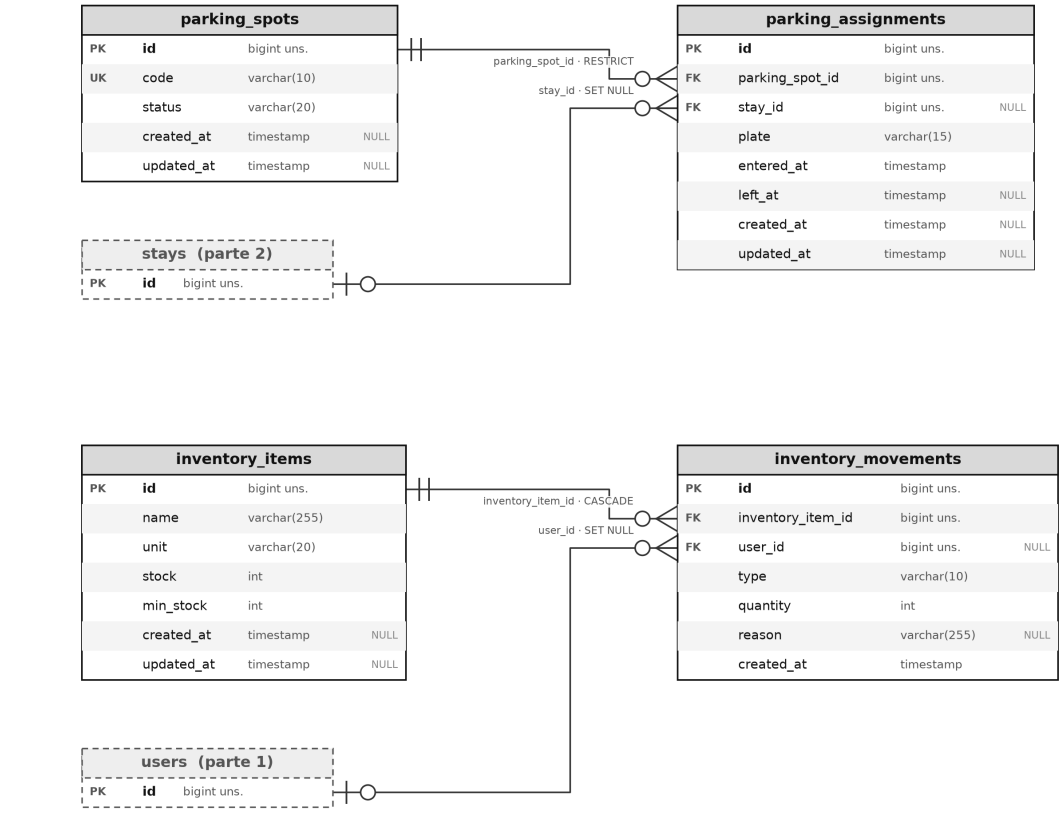


Figura 3. Diagrama de la base de datos, parte 3: cochera e inventario.

2. Endpoints definidos

Todas las rutas viven bajo /api/v1 y responden JSON. Son 77 rutas (contadas con php artisan route:list). En la tabla, «CRUD» significa listar (GET), crear (POST), ver (GET /{id}), editar (PUT/PATCH) y eliminar (DELETE).

Grupo	Método y ruta	Quién puede usarla
Salud	`GET /health`	Público
Autenticación	`POST /auth/login`	Público (5 intentos por minuto)
	`GET /auth/google/redirect · /auth/google/callback`	Público (clientes)
	`GET /auth/me · POST /auth/logout · POST /auth/refresh`	Cualquier sesión
	`POST /auth/register`	Administrador y Programador (3 por minuto)
Correo	`GET /email/verify/{id}/{hash} · POST /email/verification-notification`	Enlace firmado · sesión
Portal del cliente	`PUT /portal/profile`	Cliente
	`GET /portal/room-types`	Cliente
	`GET · POST /portal/reservations`	Cliente
	`POST /portal/reservations/{id}/cancel`	Cliente (solo las suyas)
Panel	`GET /dashboard`	Administrador, Recepcionista, Almacenero, Programador
Tipos de habitación	`GET /room-types · /room-types/{id}`	Administrador y Recepcionista
	`POST · PUT · DELETE /room-types`	Administrador
Reservas	`CRUD /reservations · POST /reservations/{id}/check-in`	Administrador y Recepcionista
Estadías	`GET /stays · /stays/{id} · POST /stays/{id}/check-out`	Administrador y Recepcionista
Habitaciones	`CRUD /rooms · GET /rooms/available · PATCH /rooms/{id}/status`	Administrador y Recepcionista
Huéspedes	`CRUD /clients`	Administrador y Recepcionista
Cochera	`CRUD /parking-spots · CRUD /parking-assignments`	Administrador y Recepcionista
Inventario	`CRUD /inventory-items · GET · POST /inventory-items/{id}/movements`	Administrador y Almacenero
Personal	`CRUD /staff`	Administrador
Usuarios	`CRUD /users · PUT /users/{id}/roles`	Administrador
Reportes	`GET /reports/occupancy · /reports/income`	Administrador
Auditoría	`GET /audit-logs`	Administrador
Programador	`GET /dev/overview · /dev/requests · /dev/errors · /dev/routes`	Programador y Administrador

2.1 Formato único de respuesta

Todas las respuestas usan App\Support\ApiResponse. Un éxito trae success y data (y meta en los listados paginados, con un máximo de 50 por página); un error trae un código estable que la web usa para decidir qué mostrar, nunca HTML ni trazas del servidor.

Respuesta de error (ejemplo real: doble reserva)

```
{
  "success": false,
  "error": {
    "code": "ROOM_ALREADY_BOOKED",
    "message": "La habitación ya está reservada en esas fechas."
  }
}
```

Código HTTP	Cuándo responde la API	Qué muestra la web
401	Sin sesión o token vencido	Pantalla «Tu sesión terminó» y vuelve al login
403	El rol no tiene permiso	Pantalla «No tienes acceso a esta sección»
404	El registro o la ruta no existe	Pantalla «No encontramos esta página»
409	Regla del negocio: doble reserva, habitación ocupada, stock insuficiente, Programador único…	Mensaje en el formulario
422	Datos inválidos (FormRequest)	Error junto a cada campo
429	Demasiados intentos (Retry-After)	Pantalla con cuenta regresiva
500	Error inesperado, con código de incidente	Pantalla 500 con el código para soporte
503	Servicio no disponible o sin conexión	Pantalla que reintenta sola

3. Registro y login diferenciados

Hay dos tipos de cuenta con caminos separados, tanto en la pantalla como en la API. Las reglas se validan en el servidor: aunque alguien modifique la página, la API rechaza lo que no corresponde.

	Personal del hotel	Cliente (huésped)
Roles	Administrador, Recepcionista, Almacenero, Programador	Cliente
Cómo entra	Correo y contraseña (POST /auth/login)	Solo con Google (OAuth); no tiene contraseña
Quién crea la cuenta	El Administrador o el Programador, desde «Registrar personal». No hay registro público	Se crea sola la primera vez que entra con Google
Datos extra	Ficha de personal: cargo, turno y fecha de ingreso	DNI (8 dígitos) y teléfono (9 dígitos), una sola vez
Protección	5 intentos de login por minuto; contraseña cifrada con bcrypt; mínimo 8 caracteres con letras y números	Google verifica el correo; un DNI no puede quedar ligado a dos cuentas
A dónde llega	Al panel de su rol	A «Completar registro» y luego a «Reservar»

El Programador es único. Ni el Administrador ni el Programador pueden crear otra cuenta de Programador, darle ese rol a otra persona, quitárselo al que existe o eliminarlo. La API responde 409 DEVELOPER_UNIQUE o DEVELOPER_LOCKED si se intenta.

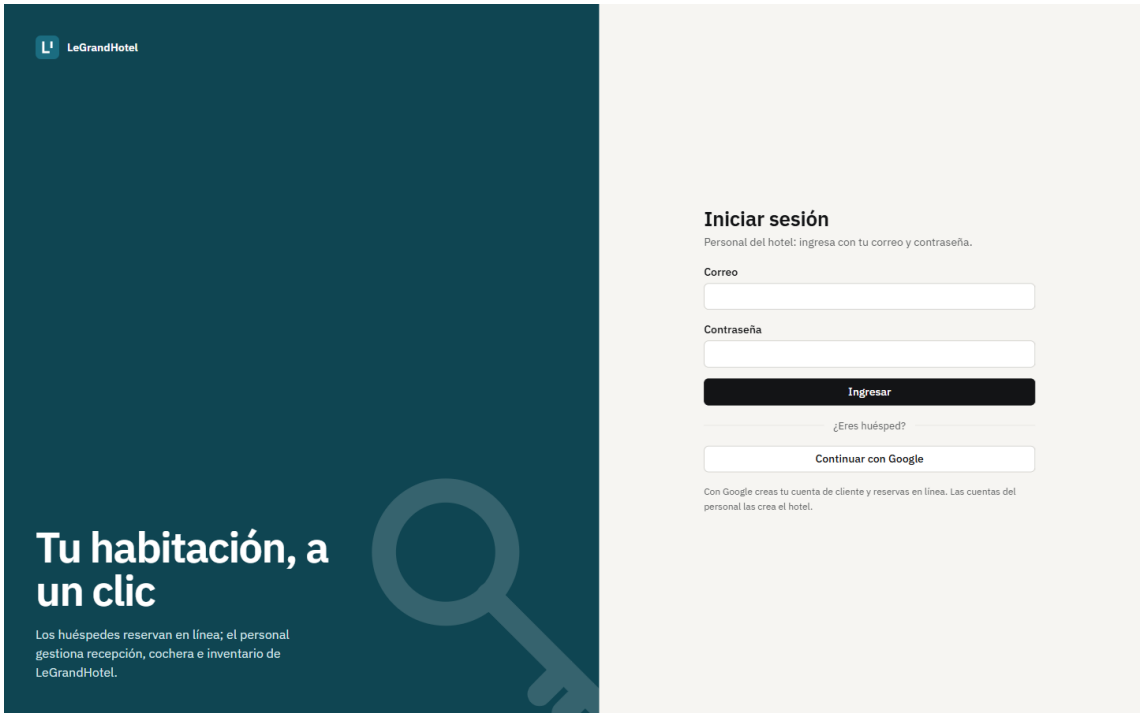


Figura 4. Login: el personal entra con correo y contraseña; el huésped, con «Continuar con Google».

3.1 Fragmentos de código

El login con contraseña solo deja entrar al personal; un cliente que intente usarlo recibe 403:

api/app/Http/Controllers/Api/AuthController.php

```
if (! $user || ! $user->password || ! Hash::check($request->string('password'), $user-
>password)) {
    throw new ApiException('INVALID_CREDENTIALS', 'Correo o contraseña incorrectos.',
401);
}
// Con contraseña solo entra el personal; los clientes entran con Google (no tienen
contraseña).
if (! $user->hasRole(...Role::INTERNAL)) {
    throw new ApiException('ACCOUNT_WITHOUT_ACCESS', 'Esta cuenta no tiene acceso al
sistema del hotel.', 403);
}
```

El registro del personal solo acepta los roles permitidos, y está protegido en la ruta por rol y por límite de intentos:

api/routes/api.php y api/app/Http/Requests/Auth/RegisterRequest.php

```
Route::post('register', [AuthController::class, 'register'])
    ->middleware(['role:admin,developer', 'throttle:register']);

'password' => ['required', 'confirmed', Password::min(8)->letters()->numbers()],
'role'     => ['required', Rule::in(Role::ASSIGNABLE)],   // Recepción, Almacén o
Administración
```

Google es solo para clientes: si el correo ya pertenece a alguien del personal, se le devuelve al login con un aviso:

api/app/Http/Controllers/Api/GoogleAuthController.php

```
$user = User::where('google_id', $googleUser->getId())->orWhere('email', $googleUser-
>getEmail())->first();
// Google es solo para clientes: el personal entra con su correo y contraseña.
if ($user && ! $user->hasRole(Role::CLIENT)) {
    return redirect()->away("{$frontend}/login?error=google_personal");
}
```

En la web, cada ruta declara qué roles la pueden abrir; si no corresponde, se muestra la pantalla 403 (y la API también responde 403):

web/src/auth/ProtectedRoute.tsx

```
export function ProtectedRoute({ roles }: { roles?: RoleName[] }) {
  const { user, hasRole } = useAuth()
  if (!user) return <Navigate to="/login" replace />
  if (roles && !hasRole(...roles)) return <ForbiddenPage />
  return <Outlet />
}
```

3.2 Pantallas de registro

Las pantallas «Registrar personal» y «Completar registro» del cliente se muestran con su descripción en las secciones 4.3 y 4.5.

4. Qué puede hacer cada rol

Cada rol ve en el menú solo sus opciones. Para cada rol se muestra primero su menú y después cada vista del menú por separado, sin la barra lateral, con la descripción de para qué sirve arriba de cada captura. Cuando una vista abre una ventana emergente (formulario), se muestra solo esa ventana. Las capturas se tomaron el 27 de octubre de 2026 con la página funcionando sobre la base MySQL del proyecto (70 habitaciones, 1000 huéspedes y 5000 reservas de demostración).

Opción del menú	Admin.	Recep.	Almacén	Program.	Cliente
Panel	✓	✓	✓	—	—
Reservas, Estadías, Habitaciones, Clientes, Cochera	✓	✓	—	—	—
Tipos de habitación	edita	ve	—	—	—
Inventario	✓	—	✓	—	—
Personal, Usuarios y roles, Reportes, Auditoría, Facturación	✓	—	—	—	—
Registrar personal	✓	—	—	✓	—
Vista del programador	✓	—	—	✓	—
Reservar, Mis reservas	—	—	—	—	✓

Si alguien escribe a mano la dirección de una opción que no está en su menú, ve la pantalla 403 (sección 5) y la API también responde 403.

4.1 Recepcionista

Atiende a los huéspedes en el día a día: reserva por ellos, hace el check-in y el check-out, controla la cochera y el estado de las habitaciones.

Menú del Recepcionista

Tiene el grupo «Inicio» con el Panel y el grupo «Recepción» con sus seis vistas de trabajo. No ve Inventario ni las opciones de Administración.

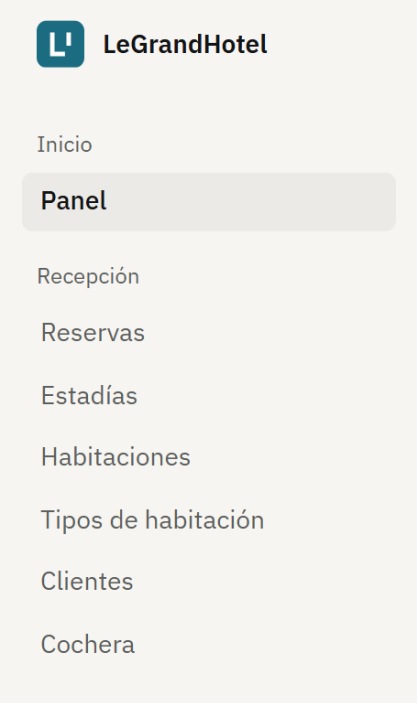


Figura 5. Menú del Recepcionista.

Panel de Recepción

Resumen del día: ocupación, habitaciones libres, llegadas y salidas de hoy y habitaciones en limpieza. A la derecha, las llegadas del día con su botón de check-in y los insumos bajo el mínimo; abajo, el tablero de habitaciones por piso con su estado.

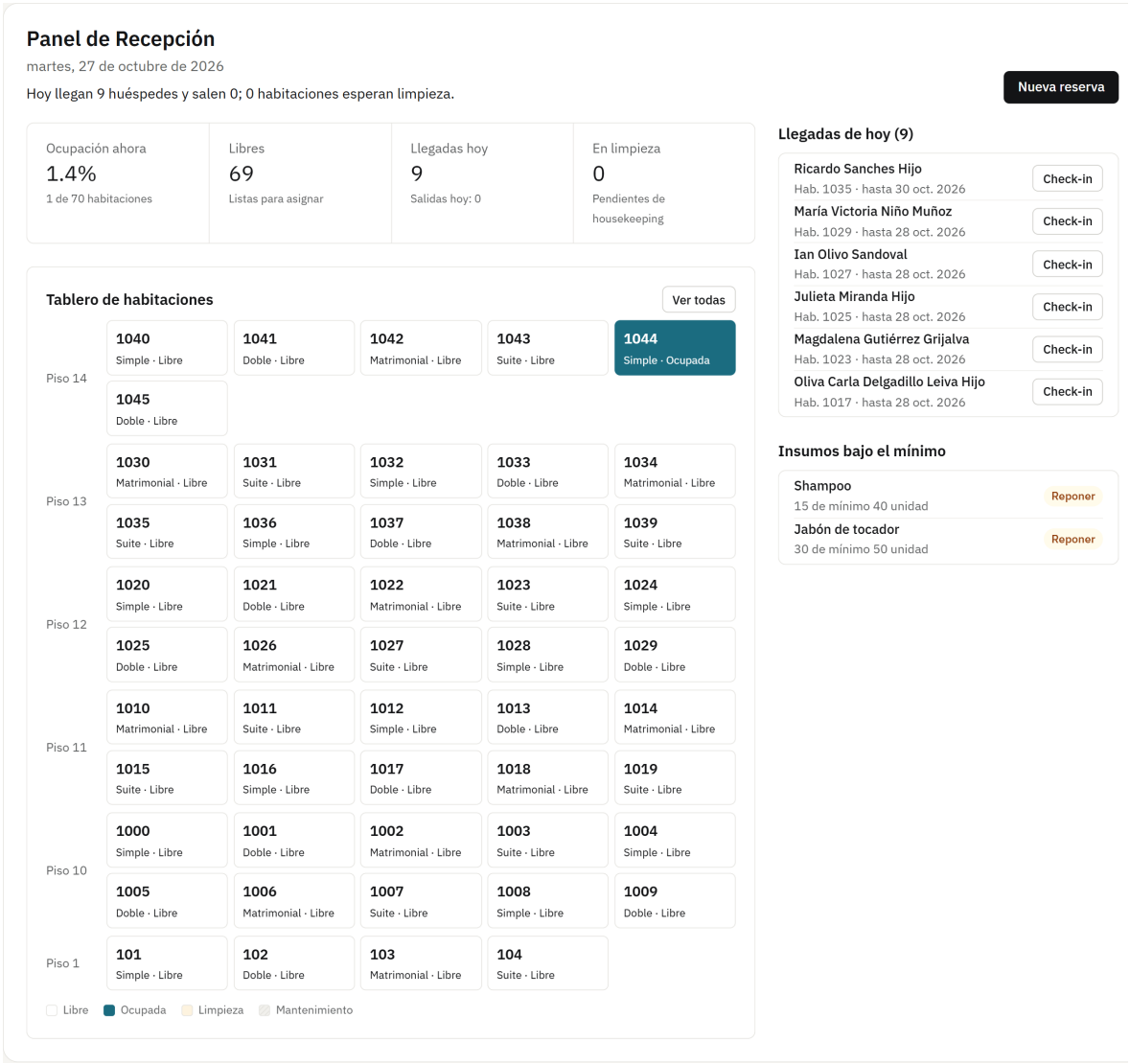


Figura 6. Panel de Recepción

Reservas

Lista de reservas paginada en el servidor, con pestañas por estado (confirmadas, por confirmar, hospedados, finalizadas y canceladas) y búsqueda por huésped o documento. Desde cada fila se hace el check-in o se cancela la reserva.

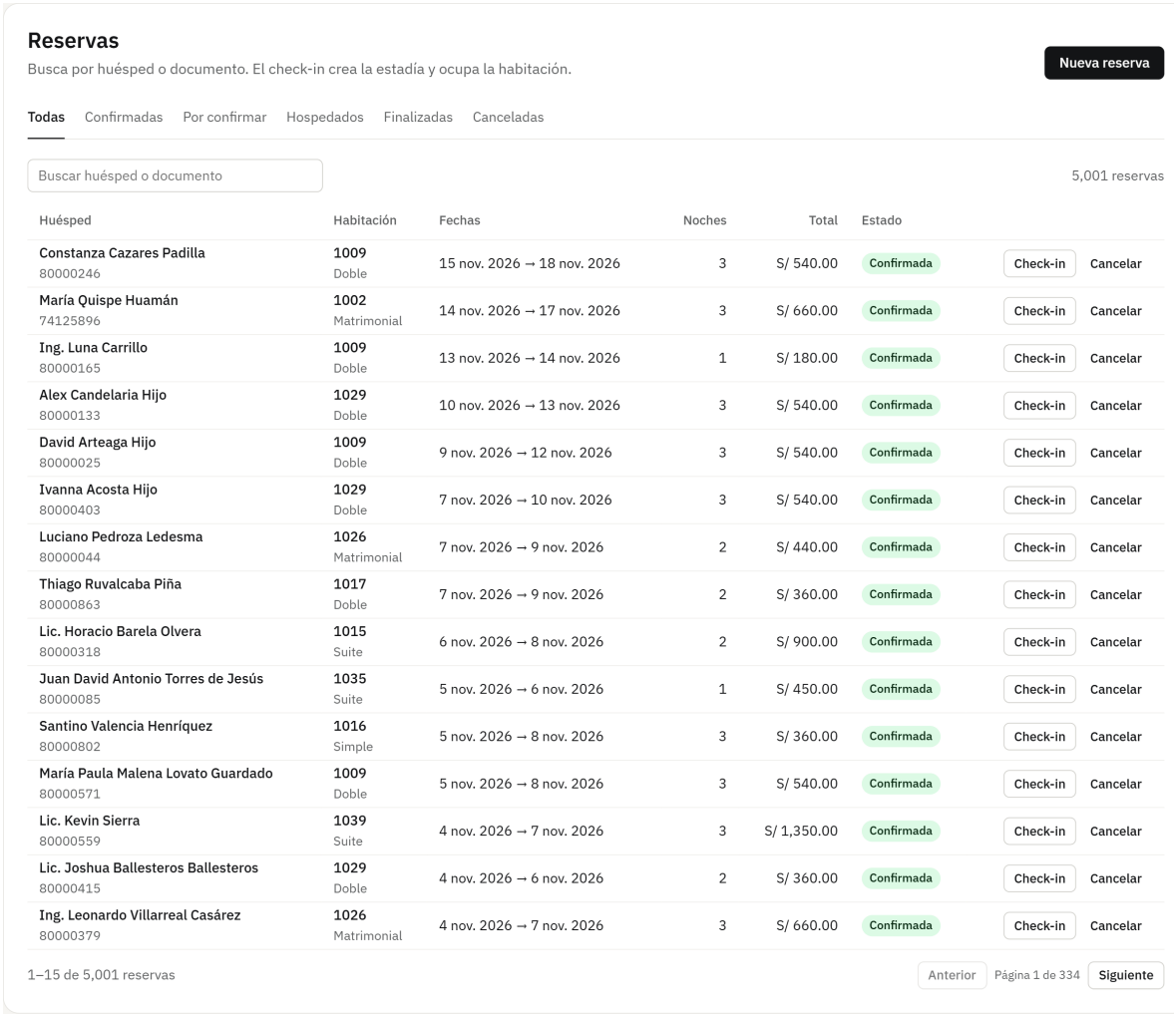


Figura 7. Reservas

Ventana emergente: Nueva reserva

Se busca al huésped, se eligen las fechas y la cantidad de huéspedes, y solo aparecen las habitaciones libres para esas fechas. El total lo calcula el servidor; si otra persona toma la misma habitación al mismo tiempo, la API responde 409.

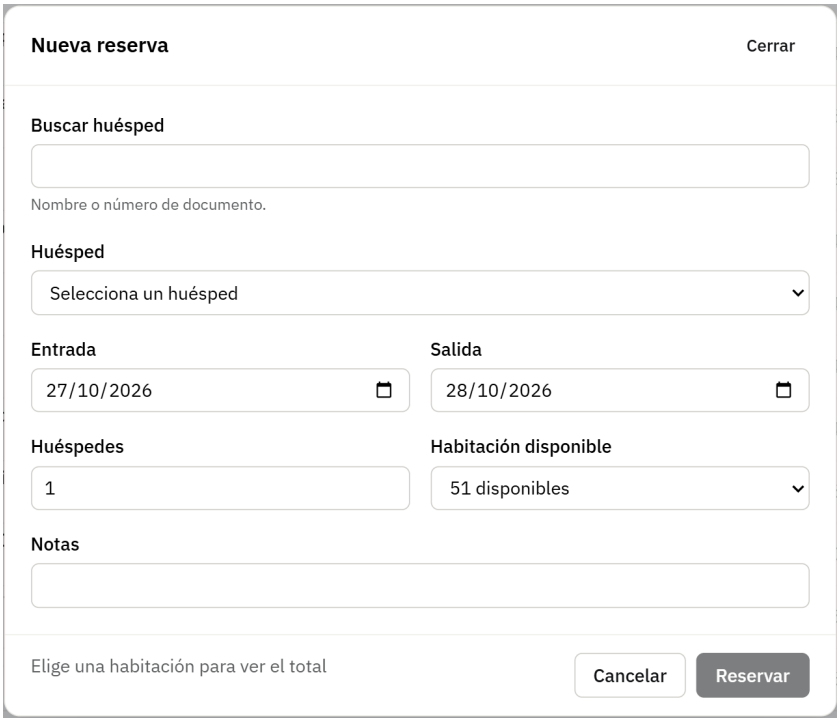


Figura 8. Ventana emergente: Nueva reserva

Estadías

Huéspedes que están dentro del hotel y el historial de estadías. El check-out cierra la estadía, pasa la habitación a limpieza y libera su espacio de cochera.

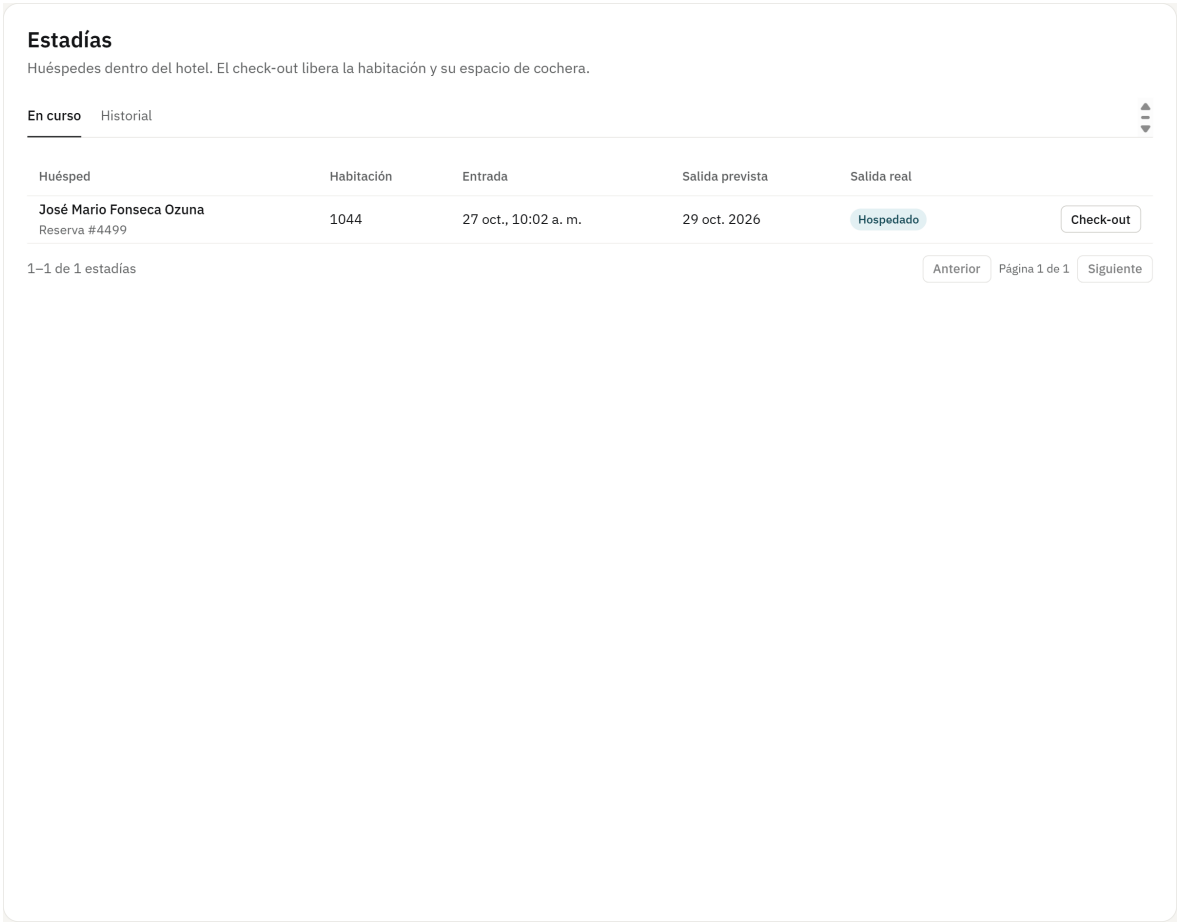


Figura 9. Estadías

Habitaciones

Habitaciones con su piso, tipo, capacidad, precio y estado, filtrables por estado. Recepción las marca como «lista» después de la limpieza.

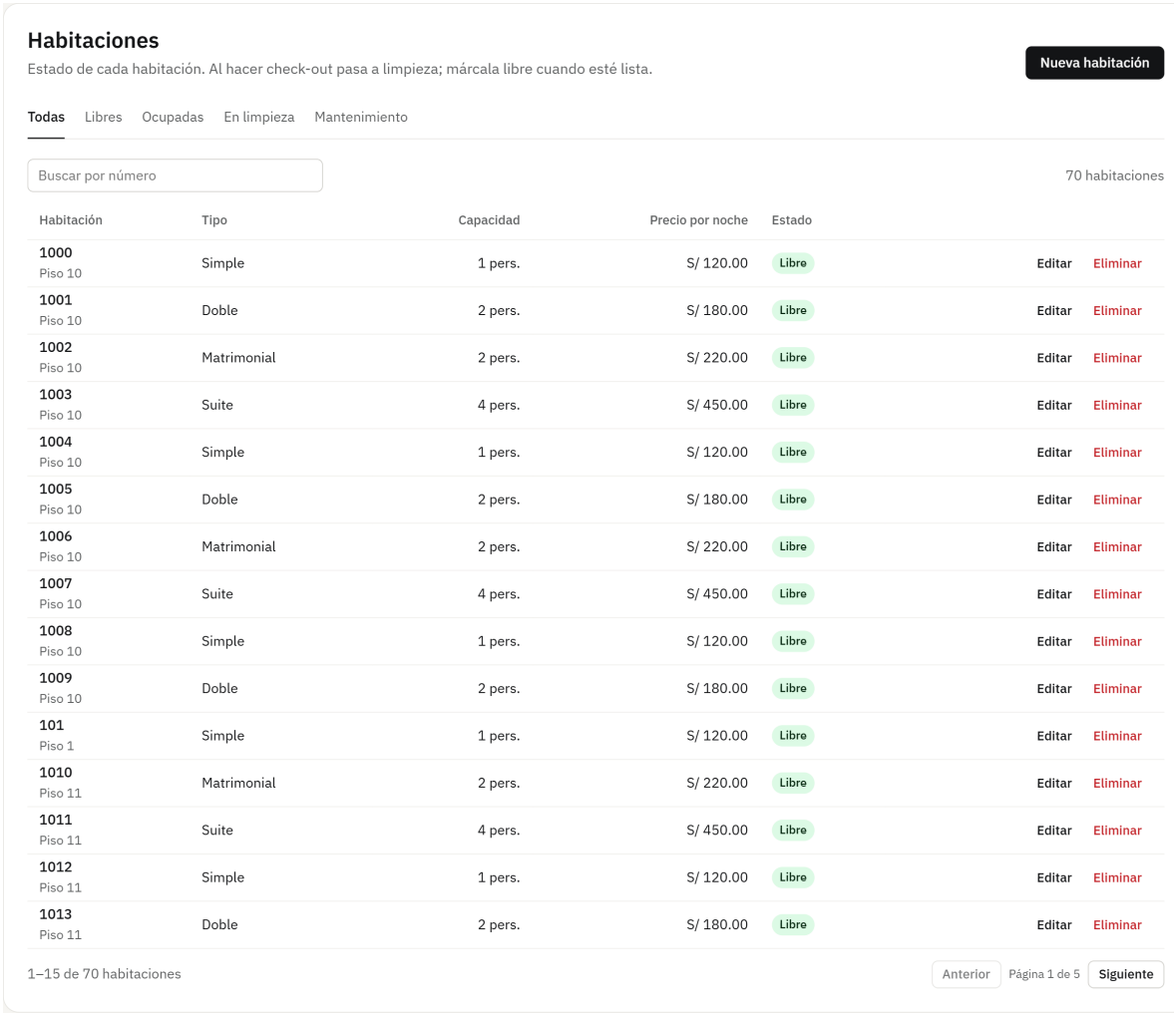


Figura 10. Habitaciones

Tipos de habitación (solo lectura)

Recepción consulta el precio por noche, la capacidad y lo que incluye cada tipo para informar al huésped, pero no puede crearlos ni cambiarlos.

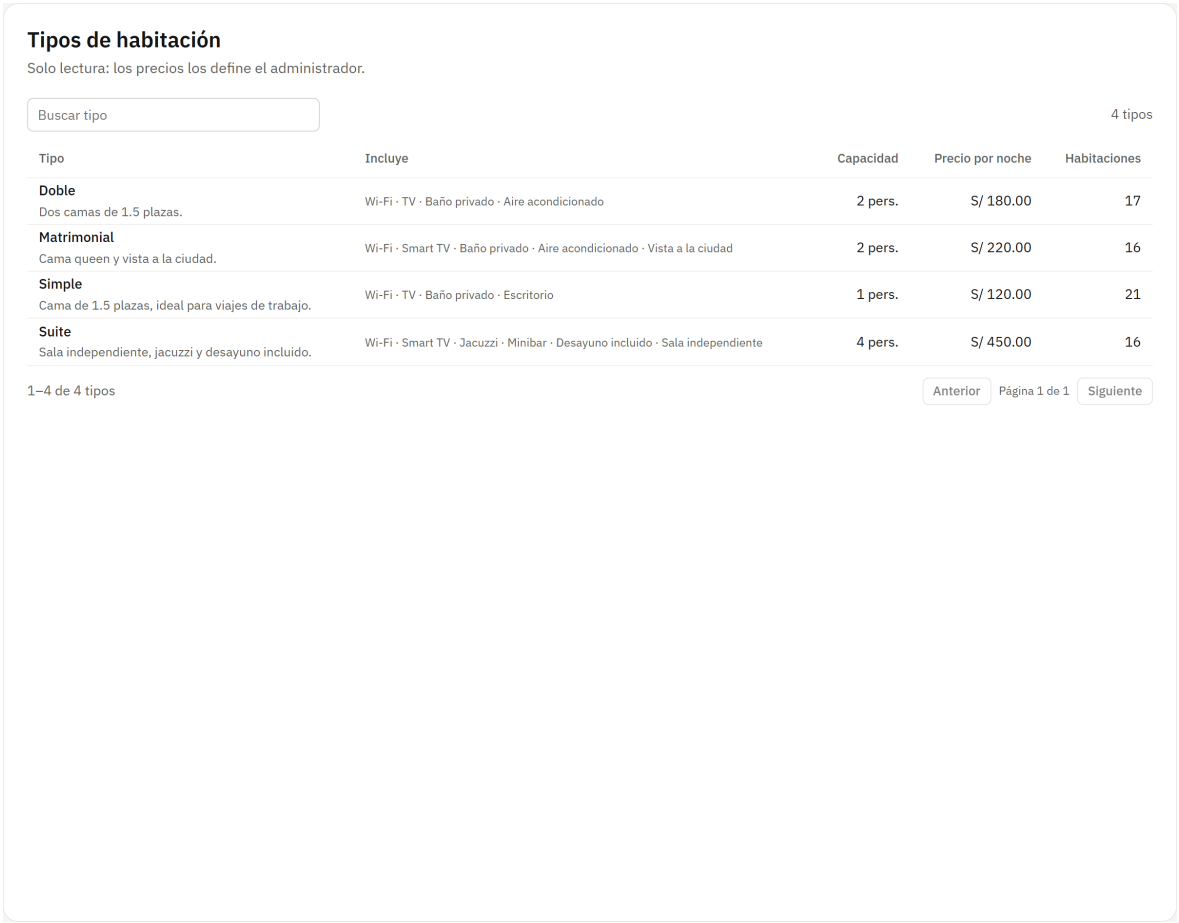


Figura 11. Tipos de habitación (solo lectura)

Clientes

Fichas de los huéspedes con su documento, teléfono, si tienen cuenta web y cuántas reservas tienen. Solo se puede eliminar un huésped sin reservas.

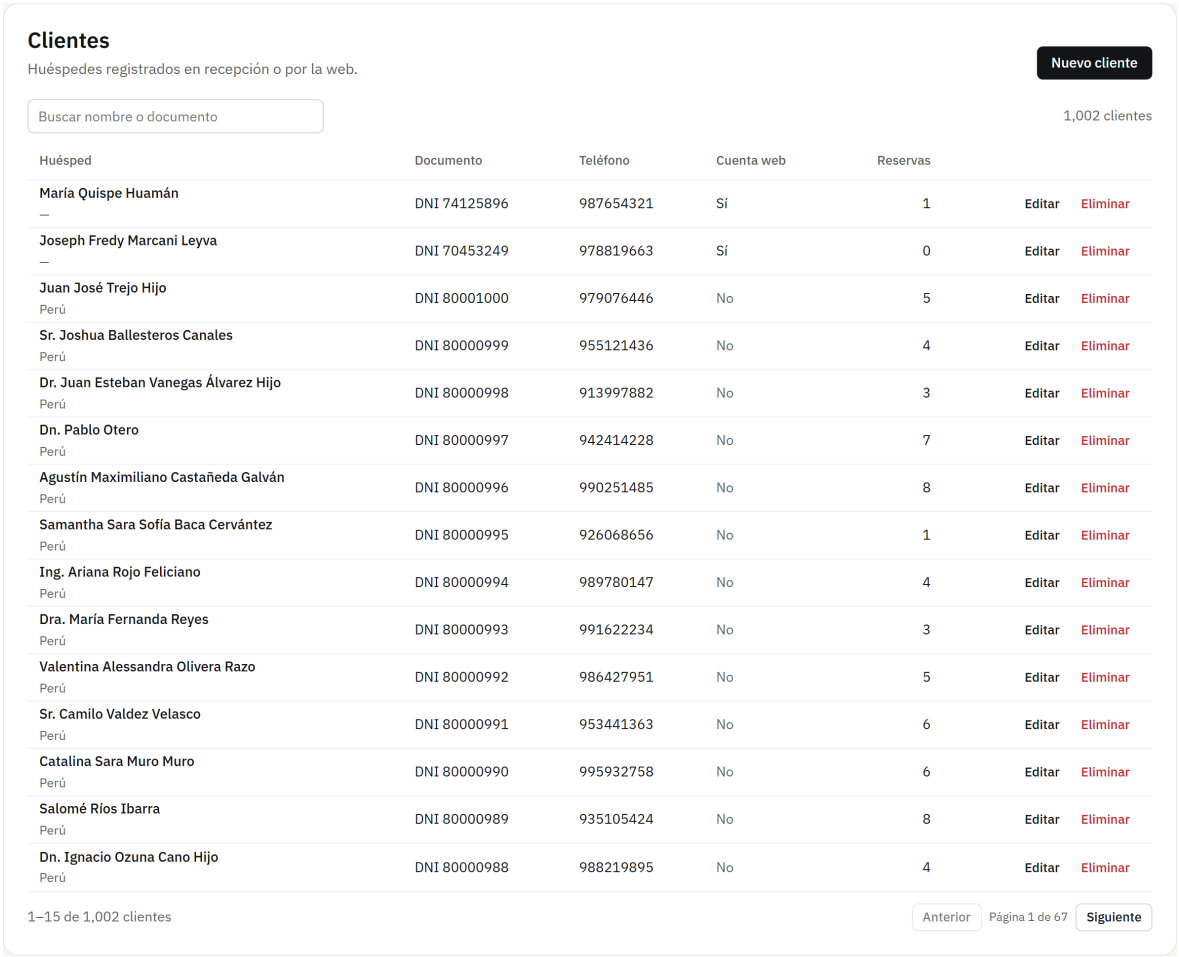


Figura 12. Clientes

Ventana emergente: Nuevo cliente

Registro de un huésped que llega o llama: nombre, tipo y número de documento (no se puede repetir), teléfono y nacionalidad.

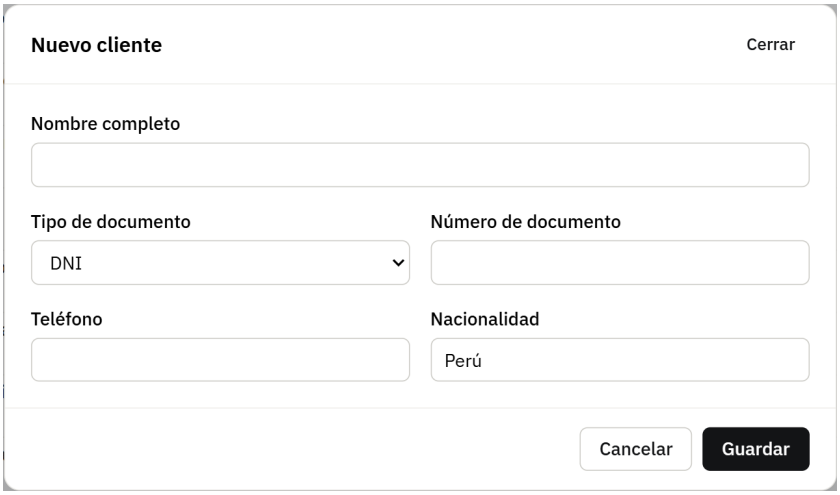


Figura 13. Ventana emergente: Nuevo cliente

Cochera

Vehículos que están dentro y los espacios de la cochera, con su estado.

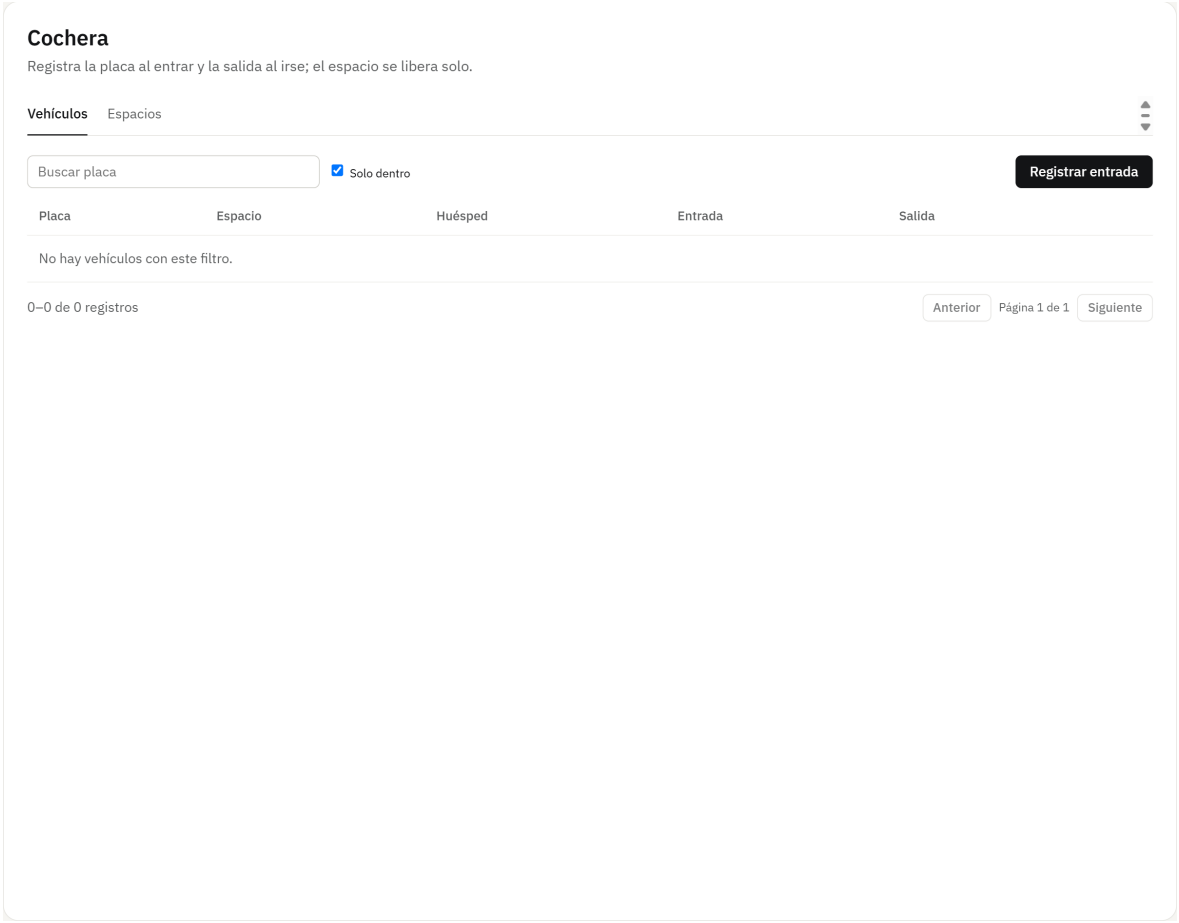


Figura 14. Cochera

Ventana emergente: Registrar entrada a cochera

Se anota la placa, el espacio libre y, si corresponde, la estadía del huésped. Si el espacio ya está ocupado, la API responde 409 PARKING_SPOT_NOT_FREE.

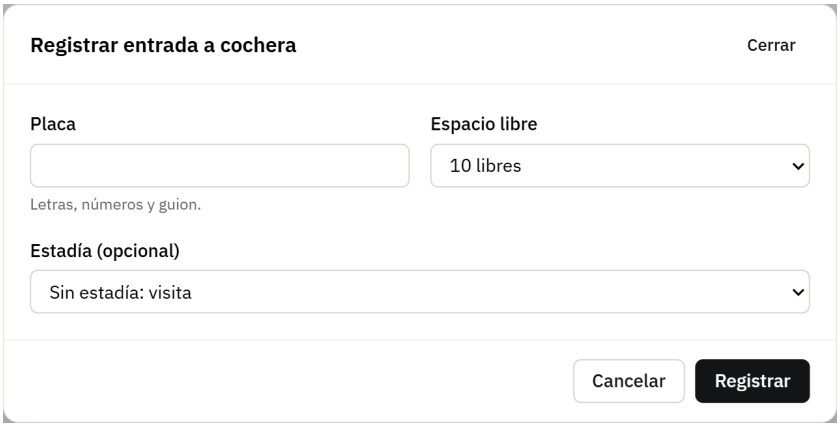


Figura 15. Ventana emergente: Registrar entrada a cochera

4.2 Almacenero

Controla los insumos del hotel: lo que entra, lo que sale y lo que hay que reponer.

Menú del Almacenero

Tiene solo dos opciones: el Panel y el Inventario.

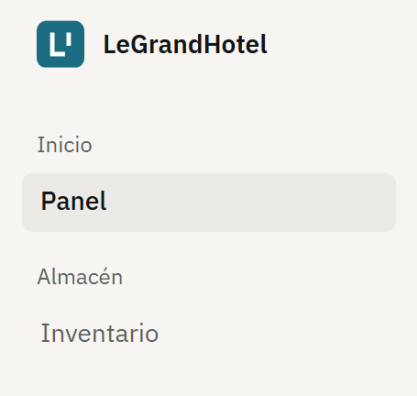


Figura 16. Menú del Almacenero.

Panel del Almacén

Muestra los insumos que están por debajo del mínimo, para reponerlos a tiempo, y el botón para registrar un movimiento.

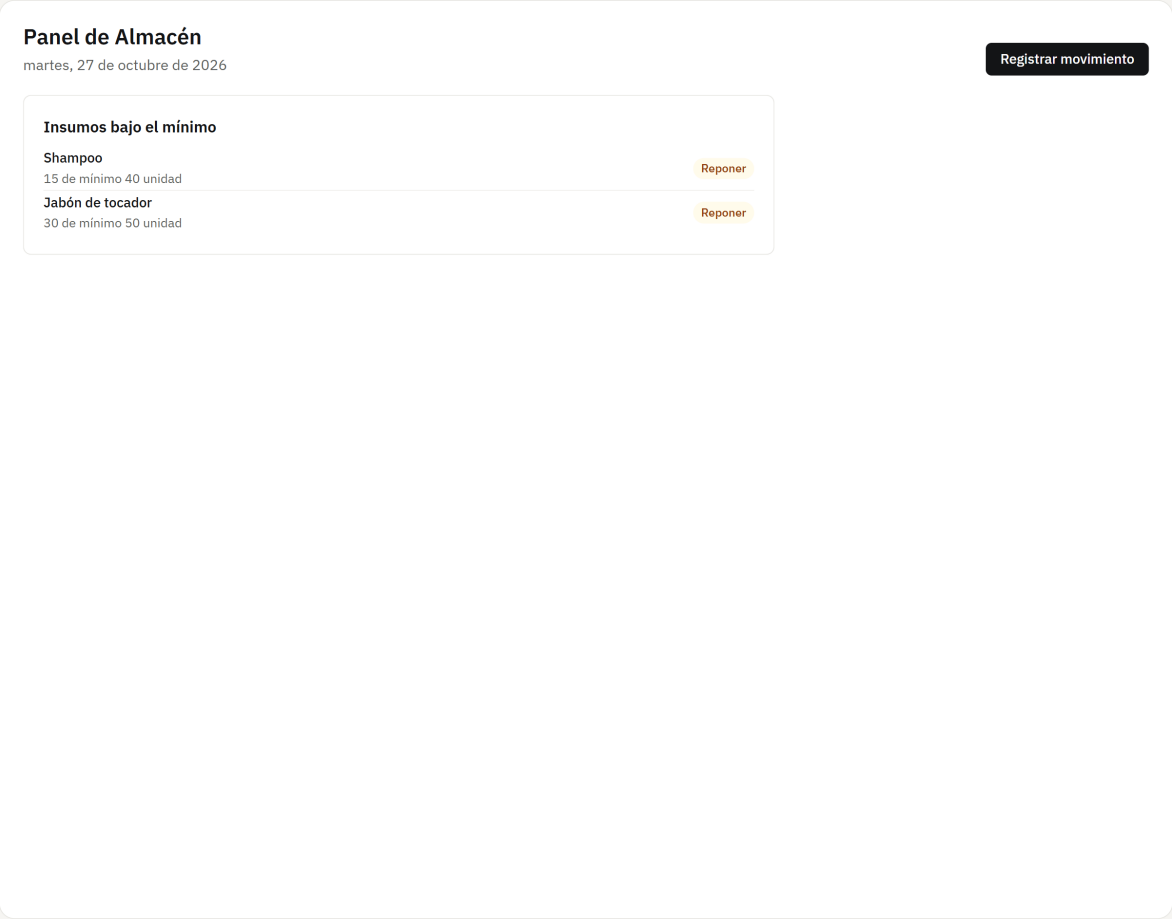


Figura 17. Panel del Almacén

Inventario

Cada insumo con su unidad, stock, mínimo y una barra de nivel; se pueden ver solo los que están bajo el mínimo. Desde cada fila se registra un movimiento o se edita el insumo.

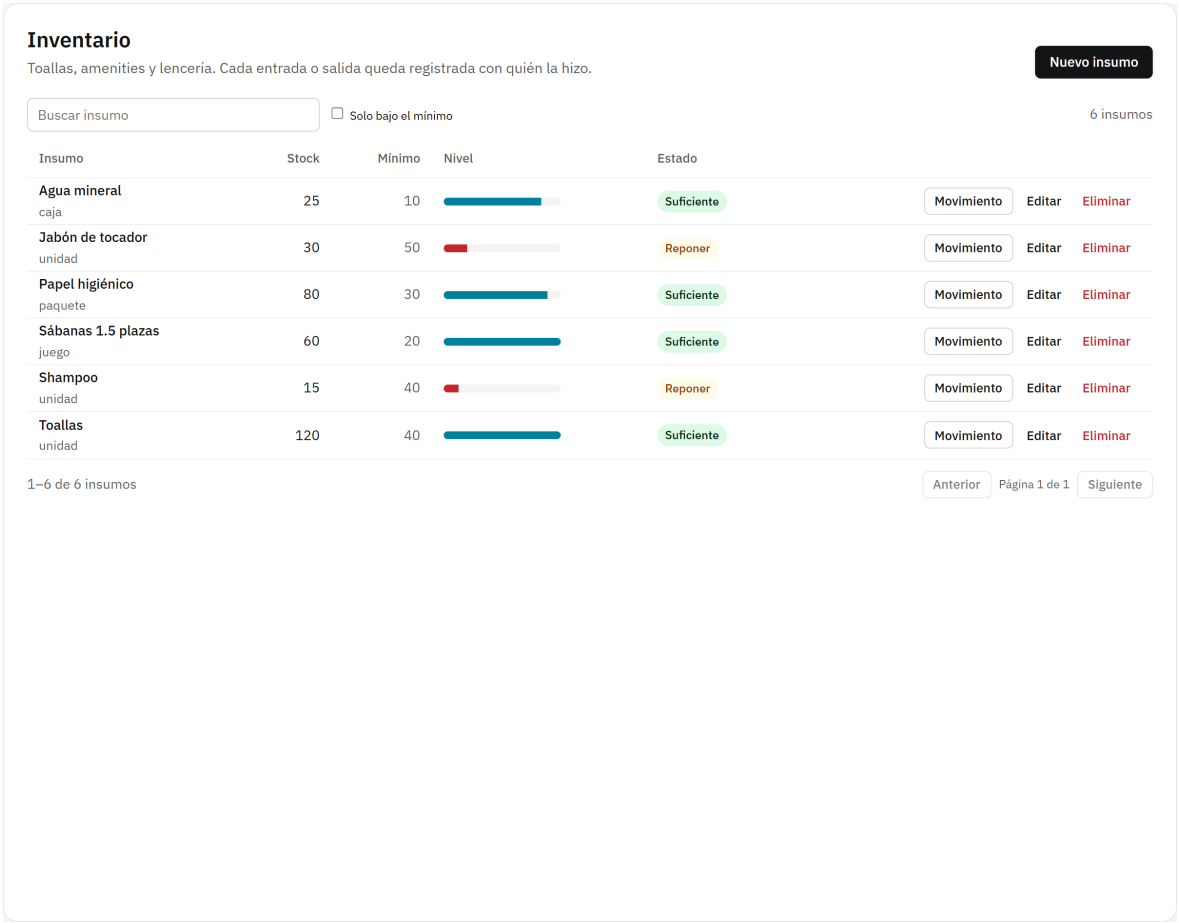


Figura 18. Inventario

Ventana emergente: Registrar movimiento

Entrada o salida con cantidad y motivo; abajo aparecen los últimos movimientos del insumo. La API no deja sacar más de lo que hay (409 INSUFFICIENT_STOCK).

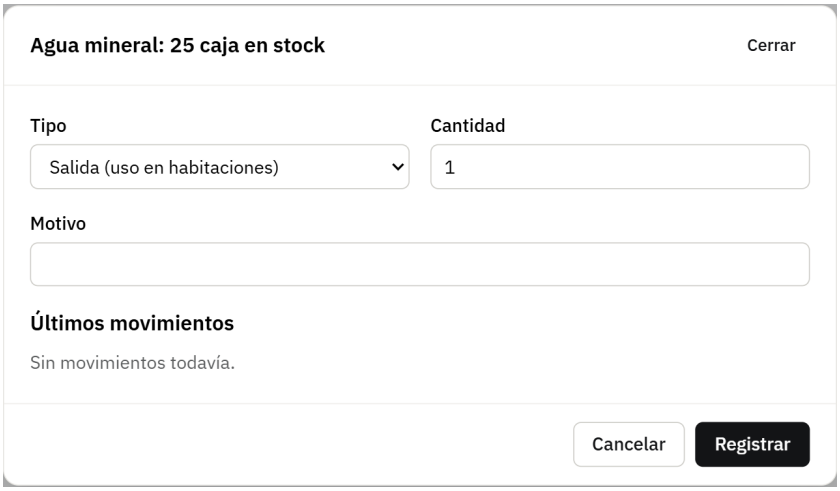


Figura 19. Ventana emergente: Registrar movimiento

Ventana emergente: Nuevo insumo

Nombre, unidad, stock inicial y mínimo. Después el stock solo cambia con movimientos, para que todo quede registrado.

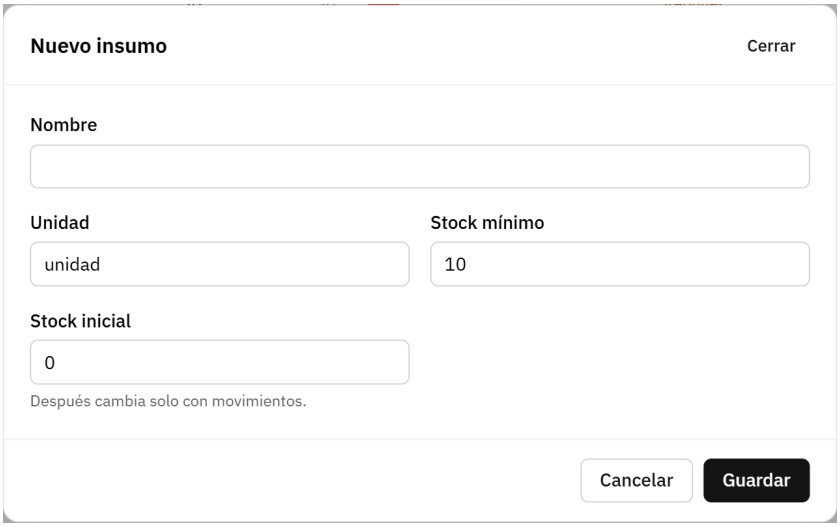


Figura 20. Ventana emergente: Nuevo insumo

4.3 Administrador

Es el responsable del sistema: tiene todas las opciones del personal, define tipos y precios, gestiona al personal y sus cuentas, y revisa reportes y auditoría.

Menú del Administrador

Es el menú completo: Inicio, Recepción, Almacén, Administración y Programador.

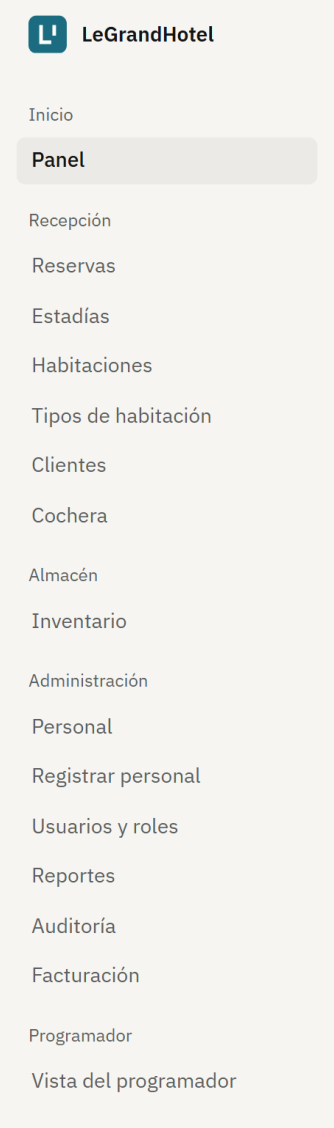


Figura 21. Menú del Administrador.

Panel del Administrador

El mismo resumen del día que Recepción, más los ingresos del mes.

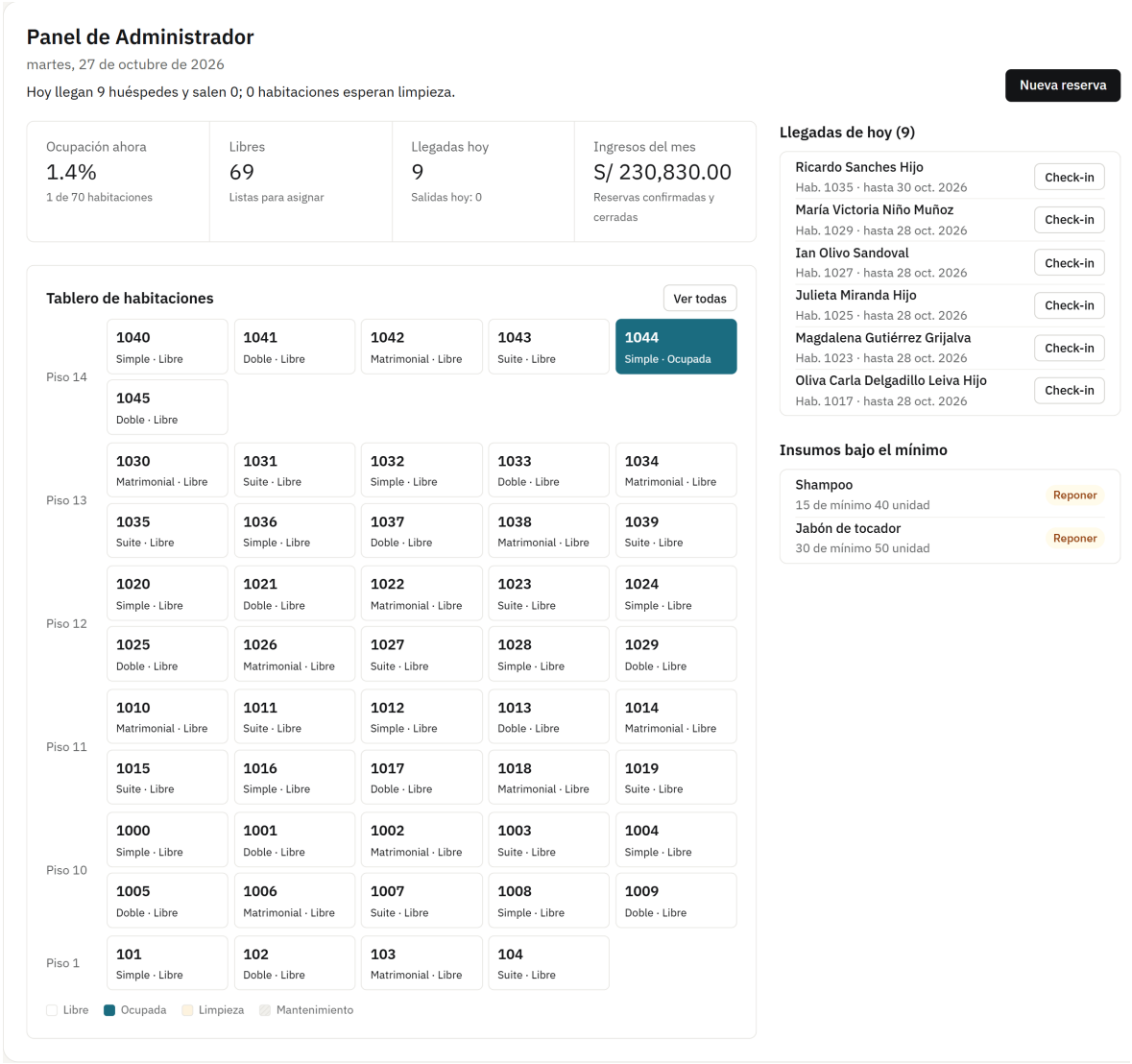


Figura 22. Panel del Administrador

Reservas

Las mismas reservas que gestiona Recepción: el Administrador también puede crear, hacer check-in y cancelar.

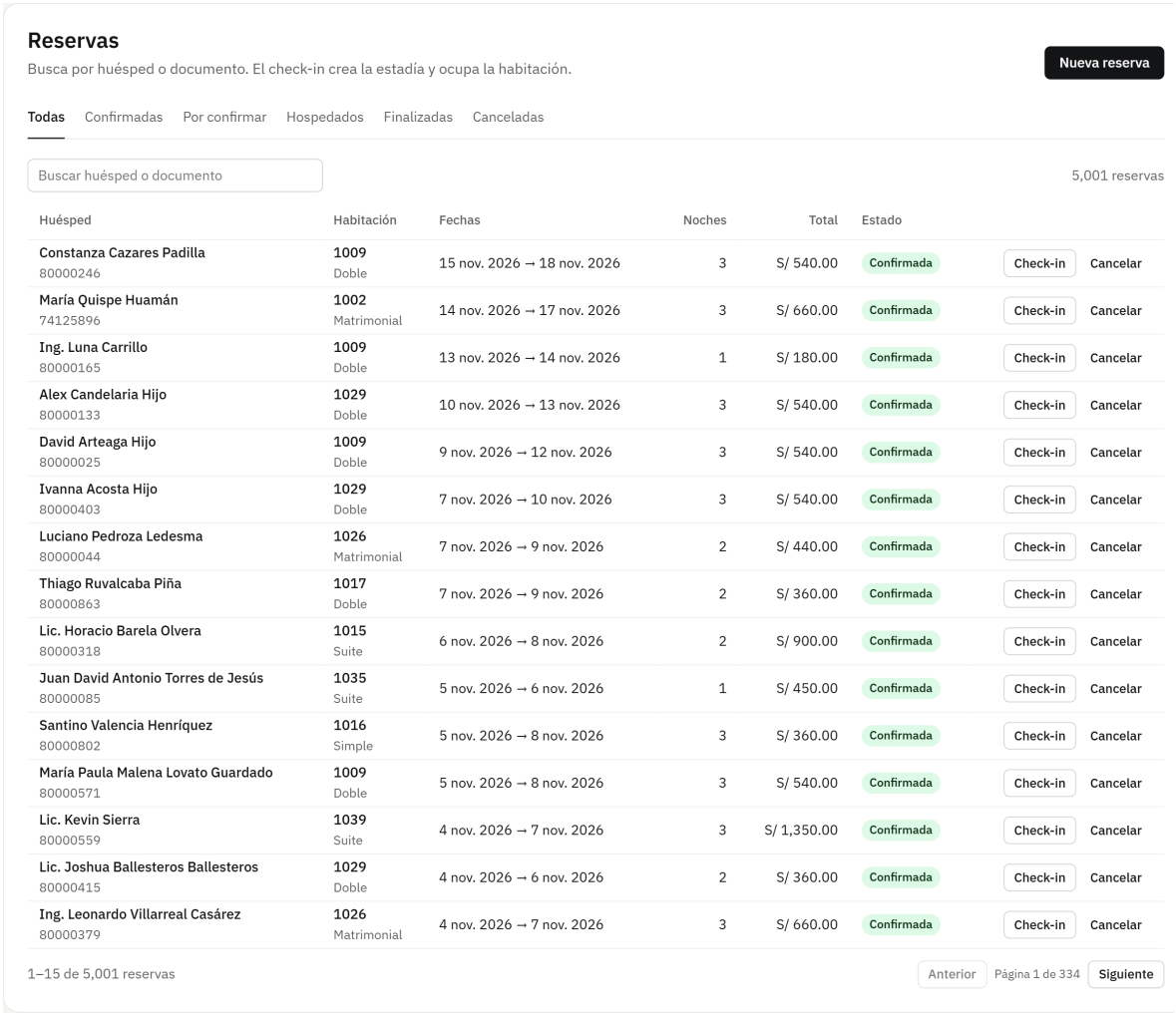


Figura 23. Reservas

Estadías

Huéspedes dentro del hotel y el historial, con el check-out.

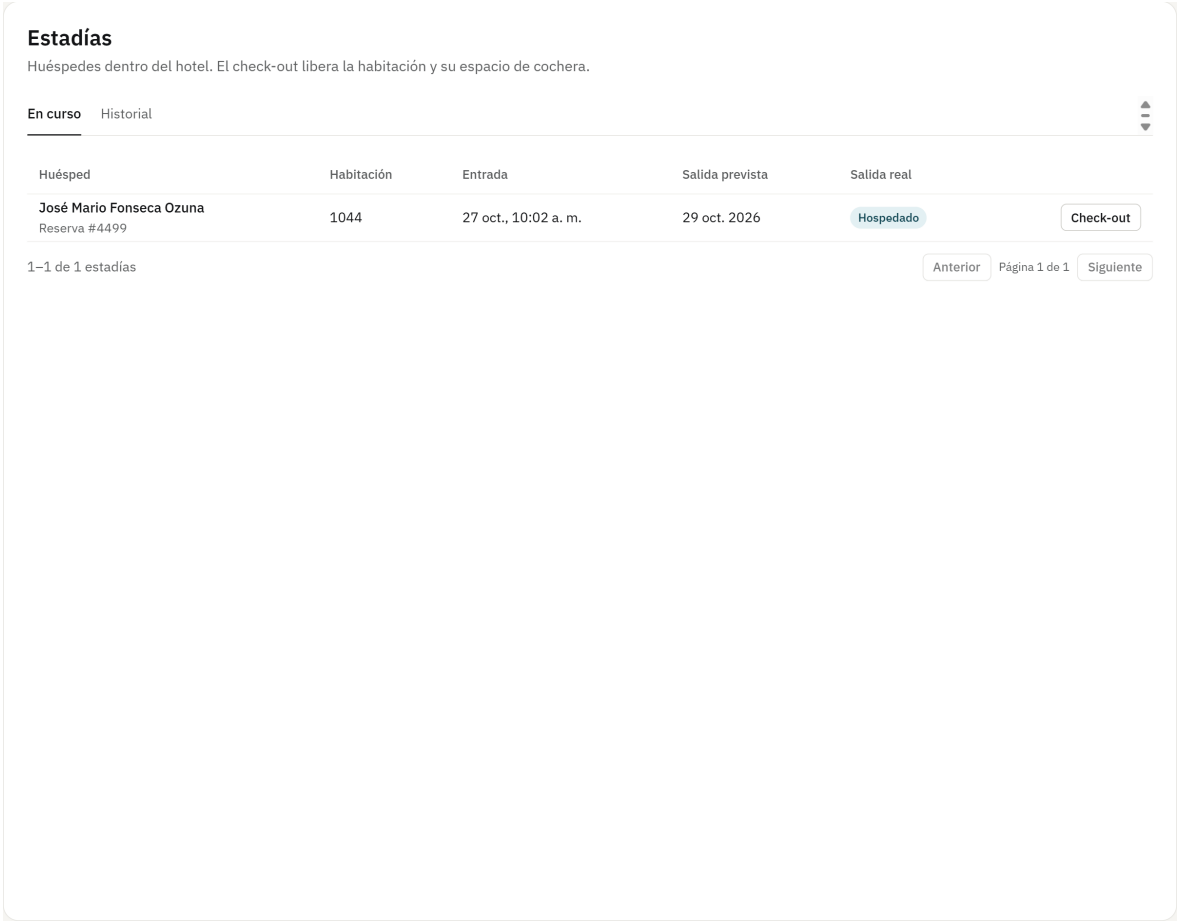


Figura 24. Estadías

Habitaciones

Habitaciones por estado; el Administrador además crea y edita habitaciones.

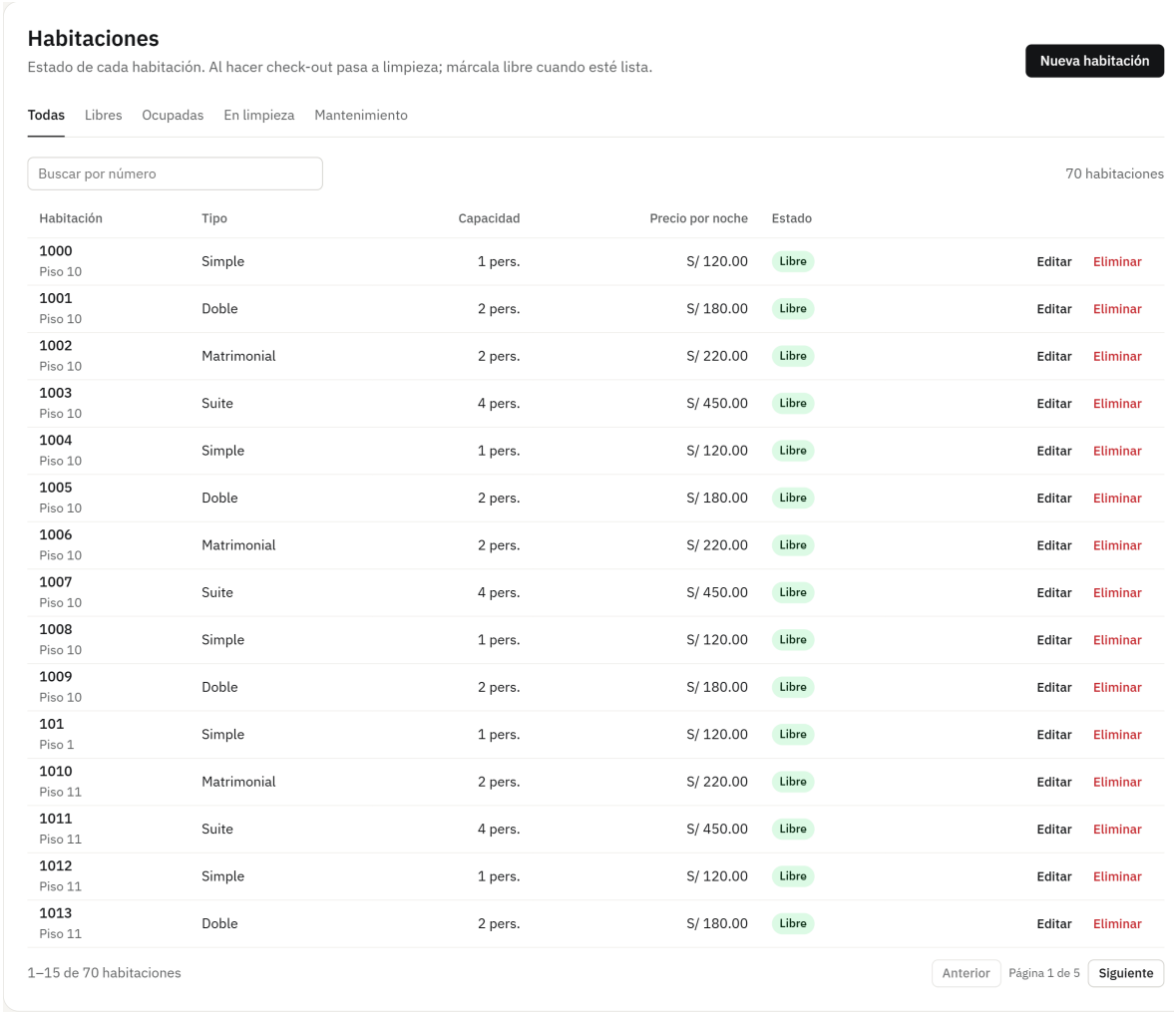


Figura 25. Habitaciones

Ventana emergente: Nueva habitación

Número (único), piso, tipo y estado inicial de la habitación.

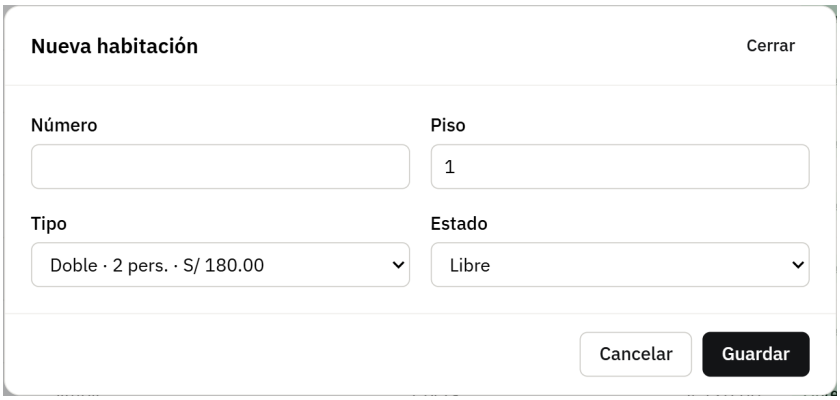


Figura 26. Ventana emergente: Nueva habitación

Tipos de habitación

A diferencia de Recepción, el Administrador crea, edita y elimina tipos.

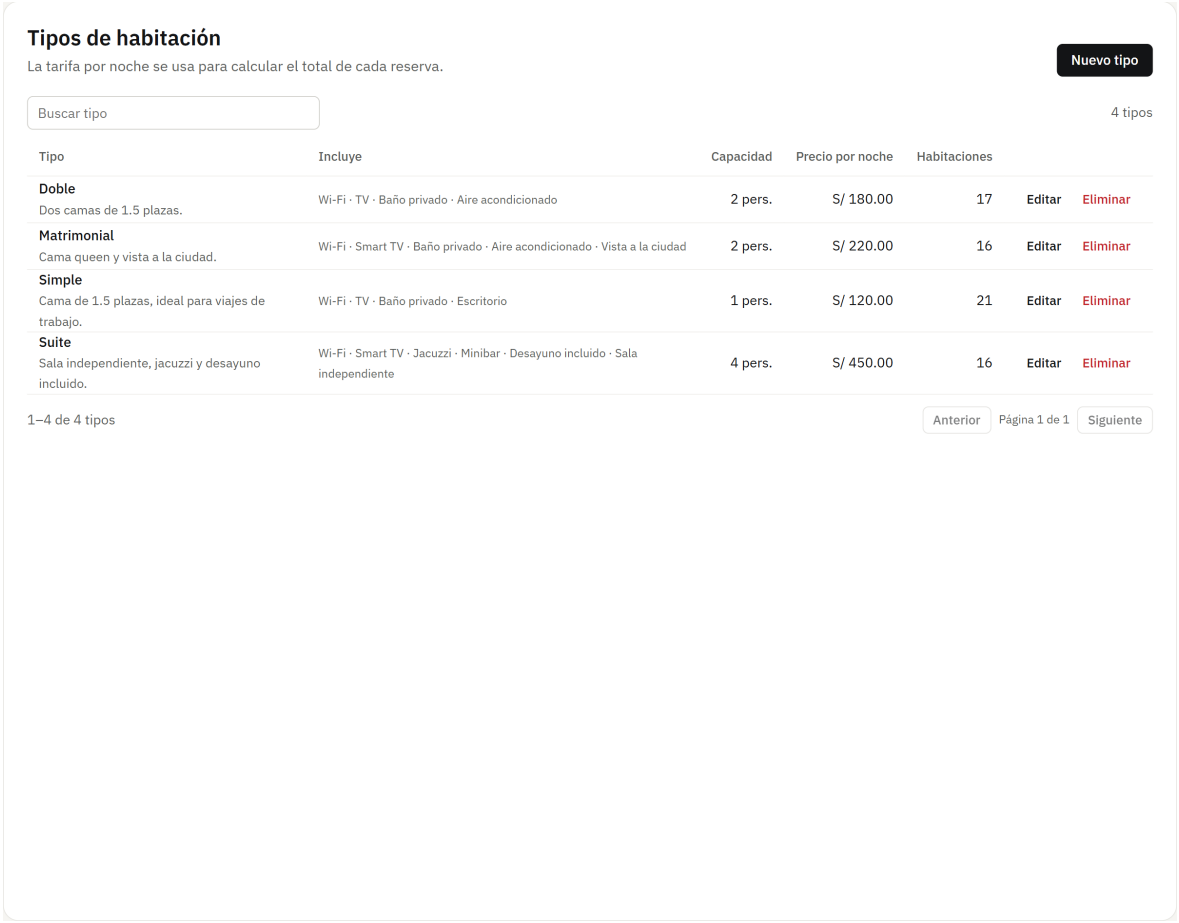


Figura 27. Tipos de habitación

Ventana emergente: Nuevo tipo de habitación

Nombre, capacidad, precio por noche, descripción y lo que incluye, separado por comas. Es lo que verá el cliente al reservar.

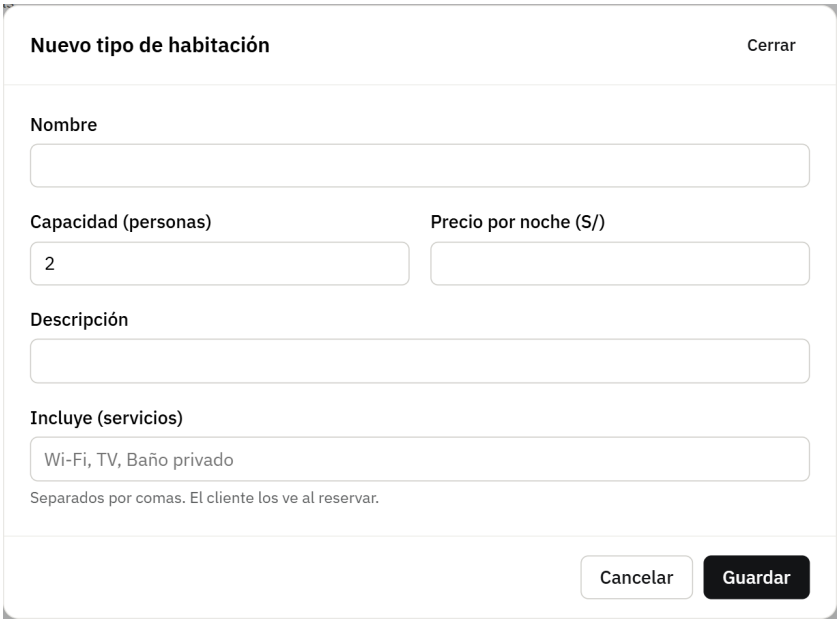


Figura 28. Ventana emergente: Nuevo tipo de habitación

Clientes

Las fichas de huéspedes, igual que en Recepción.

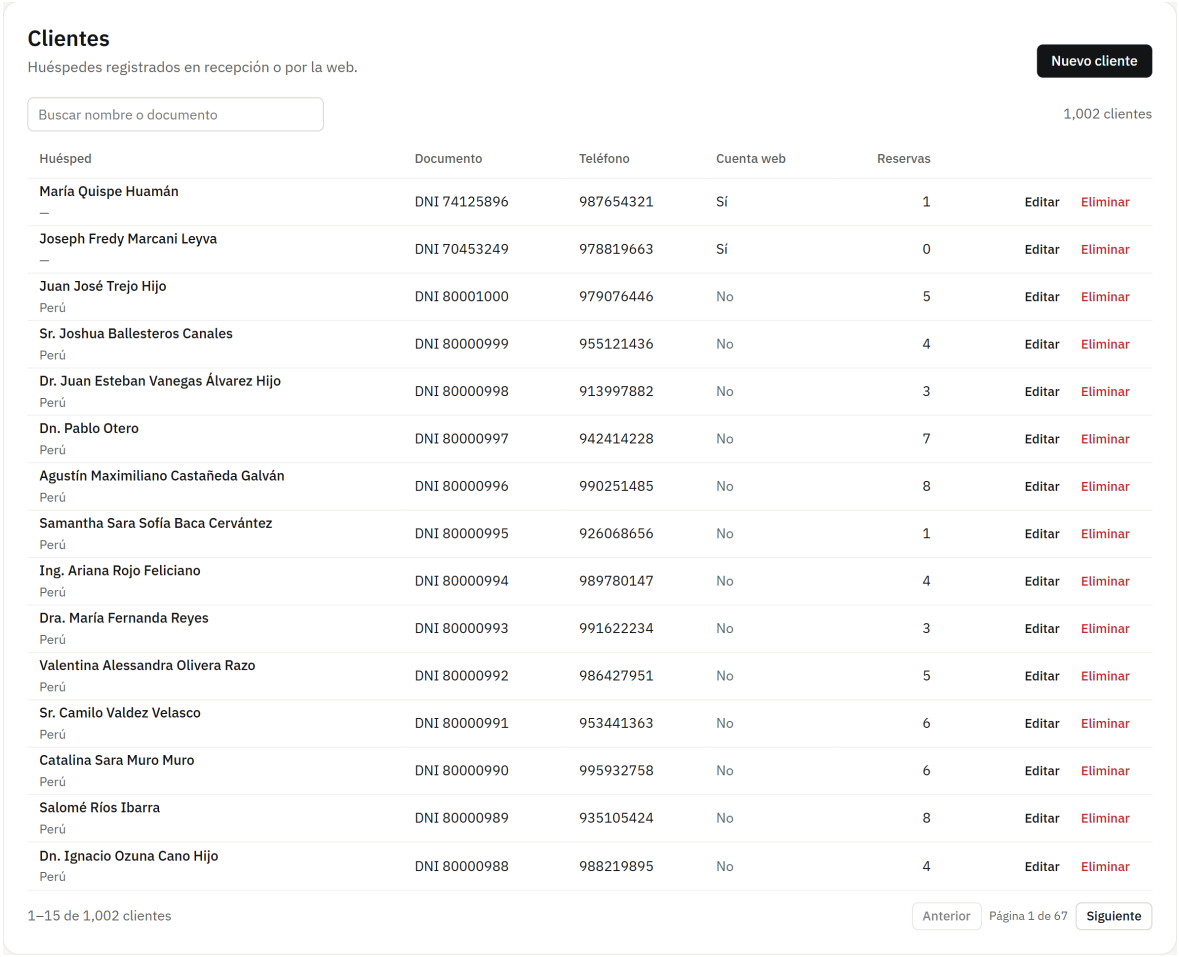


Figura 29. Clientes

Cochera

Vehículos y espacios; el Administrador también agrega espacios.

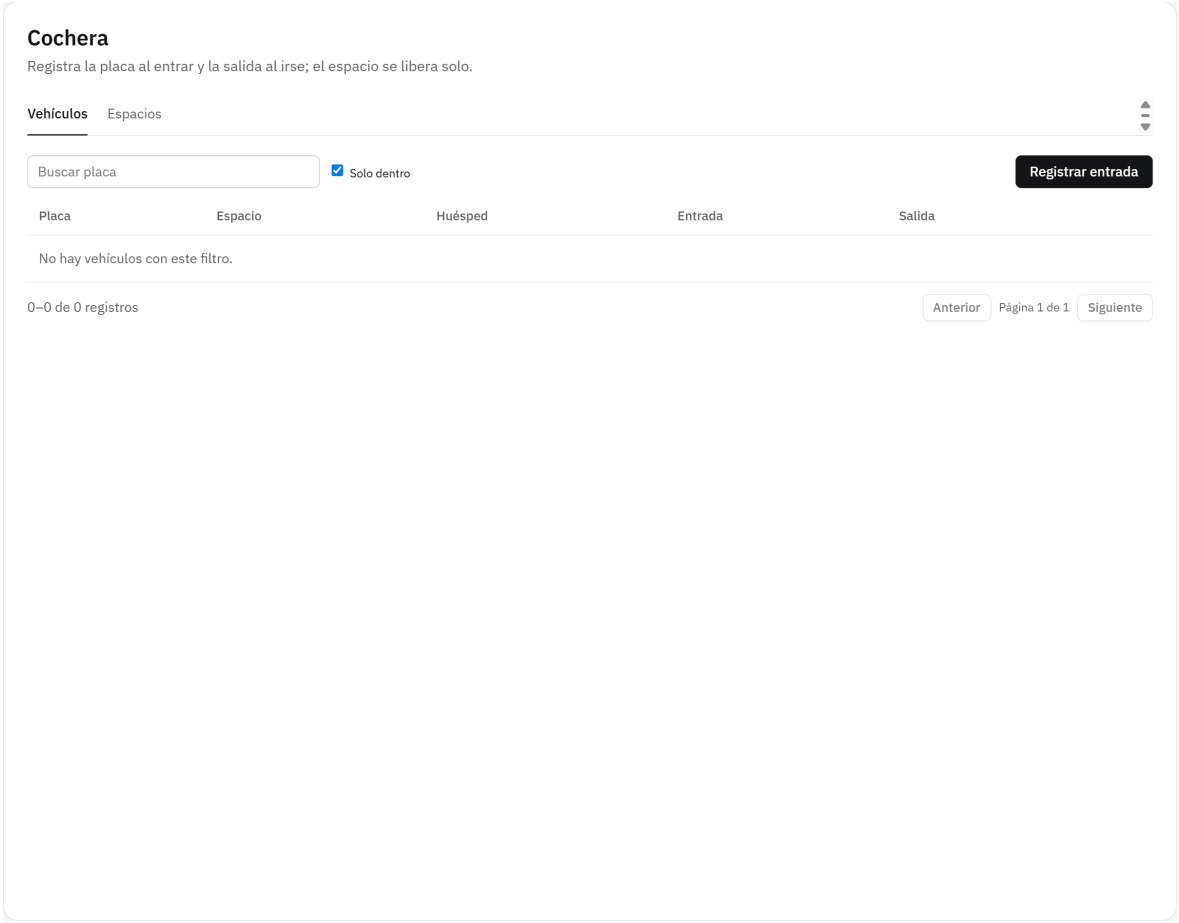


Figura 30. Cochera

Inventario

El mismo inventario del Almacenero, con los mismos permisos.

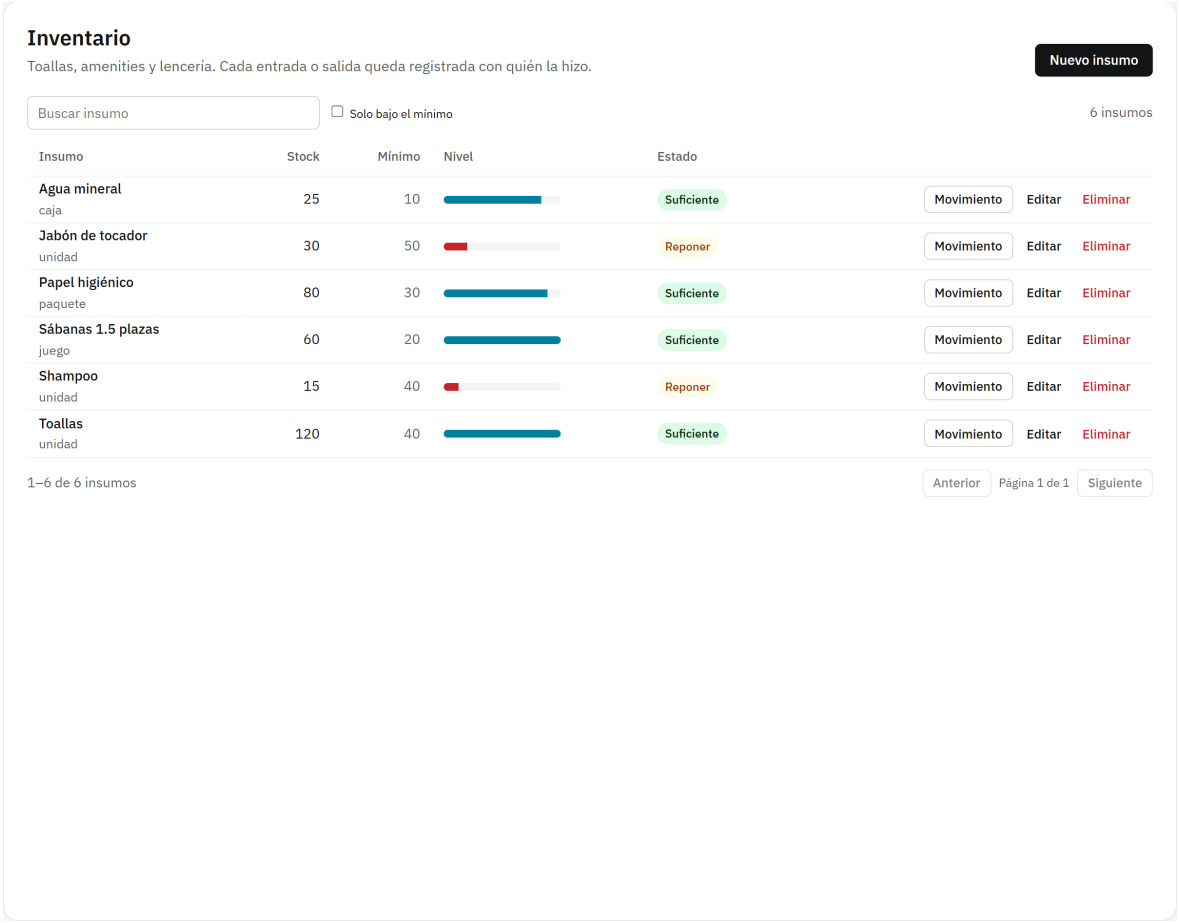


Figura 31. Inventario

Personal

Fichas del personal con su cargo, turno y fecha de ingreso, cada una ligada a su cuenta.

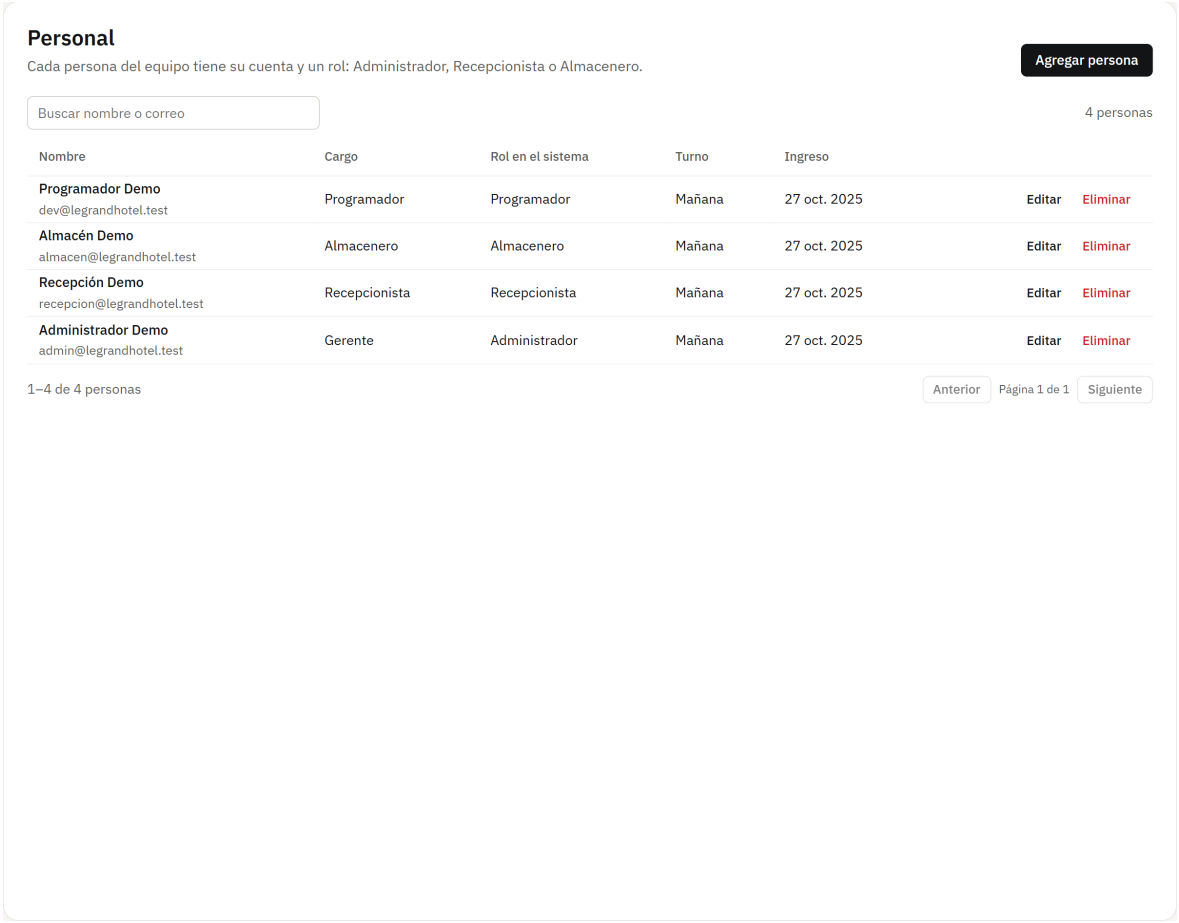


Figura 32. Personal

Ventana emergente: Agregar persona

Crea la ficha y la cuenta en un solo paso. El rol Programador no aparece porque el hotel tiene uno solo.

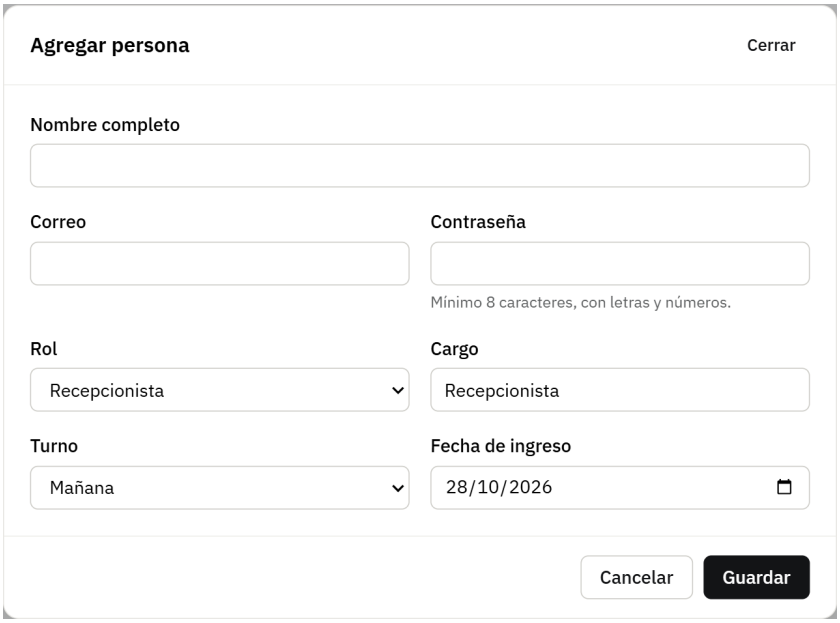


Figura 33. Ventana emergente: Agregar persona

Registrar personal

Crea la cuenta de una persona del hotel con su contraseña inicial. Solo ofrece Recepcionista, Almacenero y Administrador; la cuenta nueva recibe un correo para verificarse.

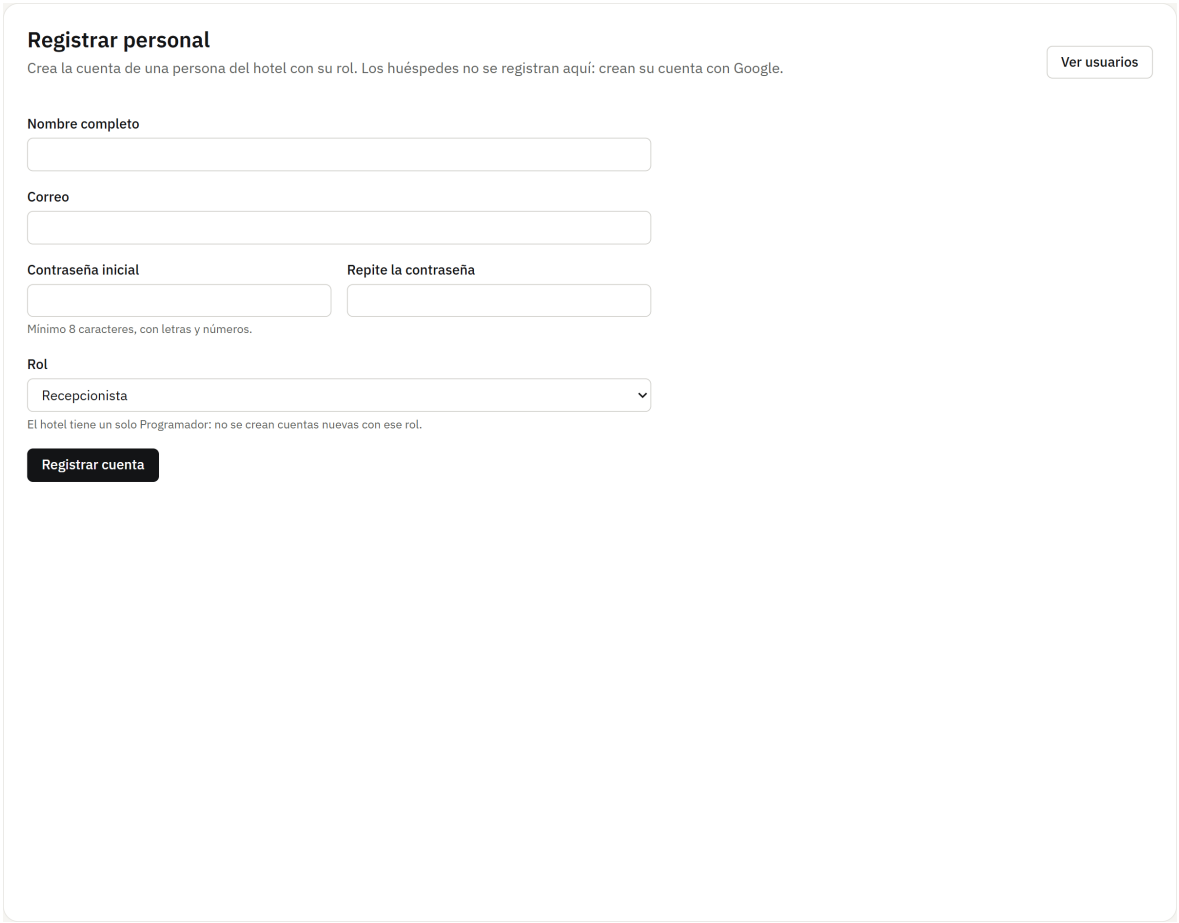


Figura 34. Registrar personal

Usuarios y roles

Todas las cuentas con su rol y si verificaron el correo, con filtros por rol y búsqueda por nombre o correo.

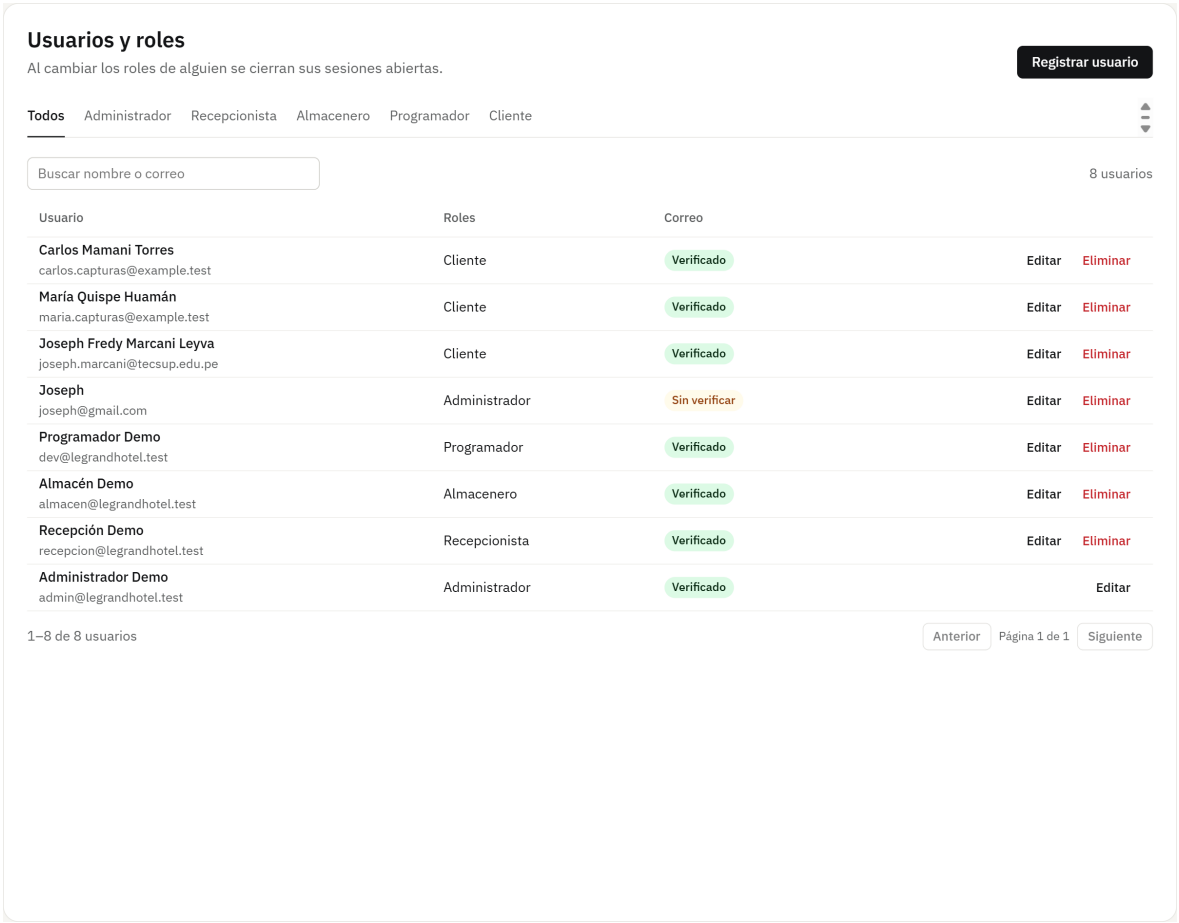


Figura 35. Usuarios y roles

Ventana emergente: Editar la cuenta del Programador

La casilla Programador está bloqueada: no se puede dar ese rol a otra cuenta ni quitárselo al Programador. Sí se pueden cambiar su nombre, su correo y su contraseña.

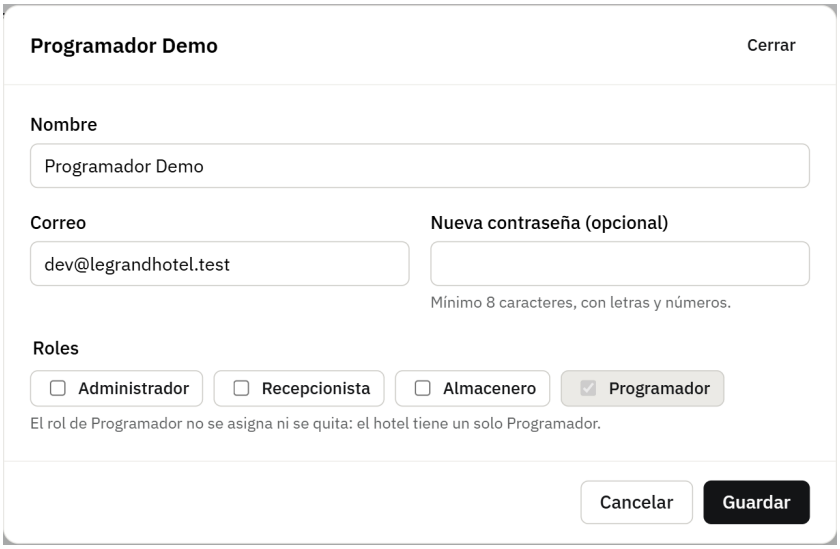


Figura 36. Ventana emergente: Editar la cuenta del Programador

Reportes

Ocupación de un rango de fechas e ingresos de los últimos 6 meses en un gráfico de barras.

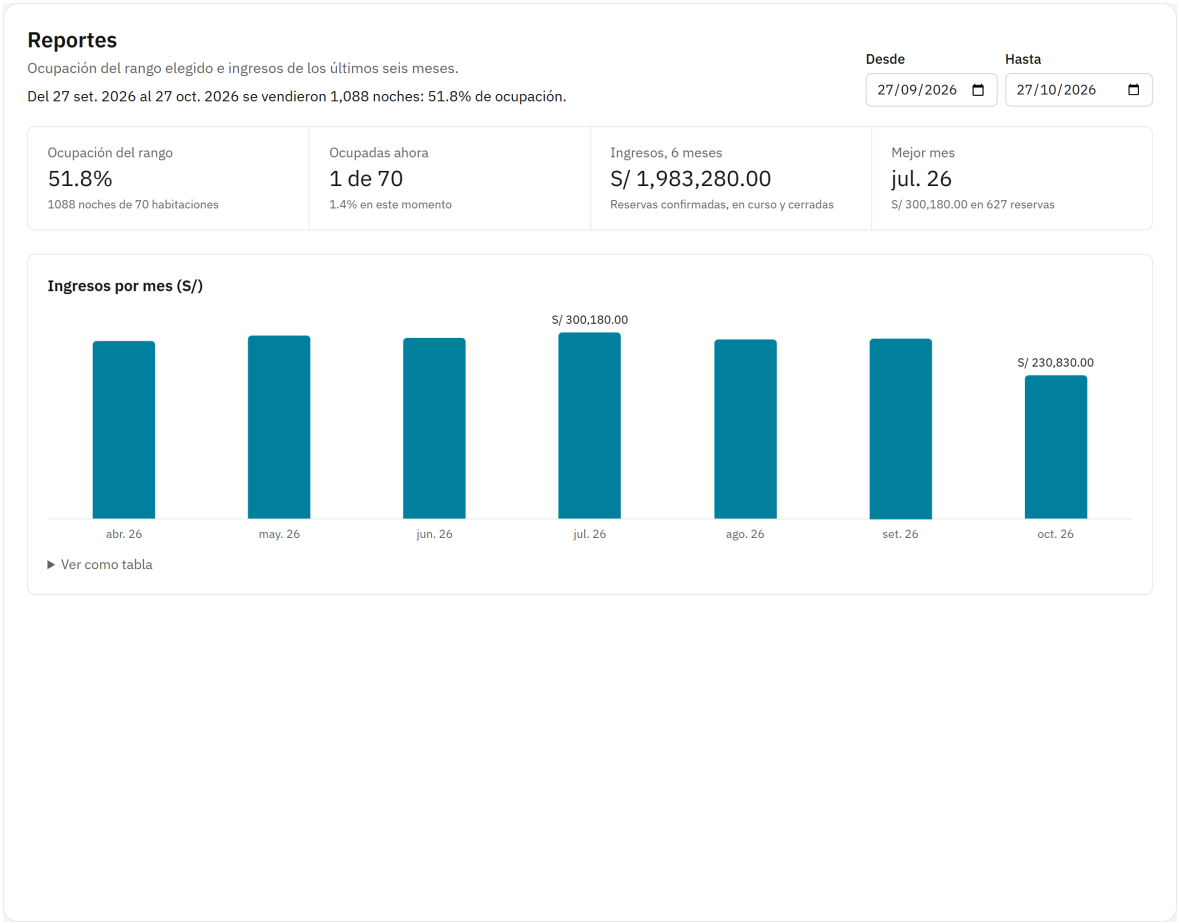


Figura 37. Reportes

Auditoría

Quién creó, editó o eliminó cada registro, cuándo y desde qué IP, con los valores antes y después del cambio.

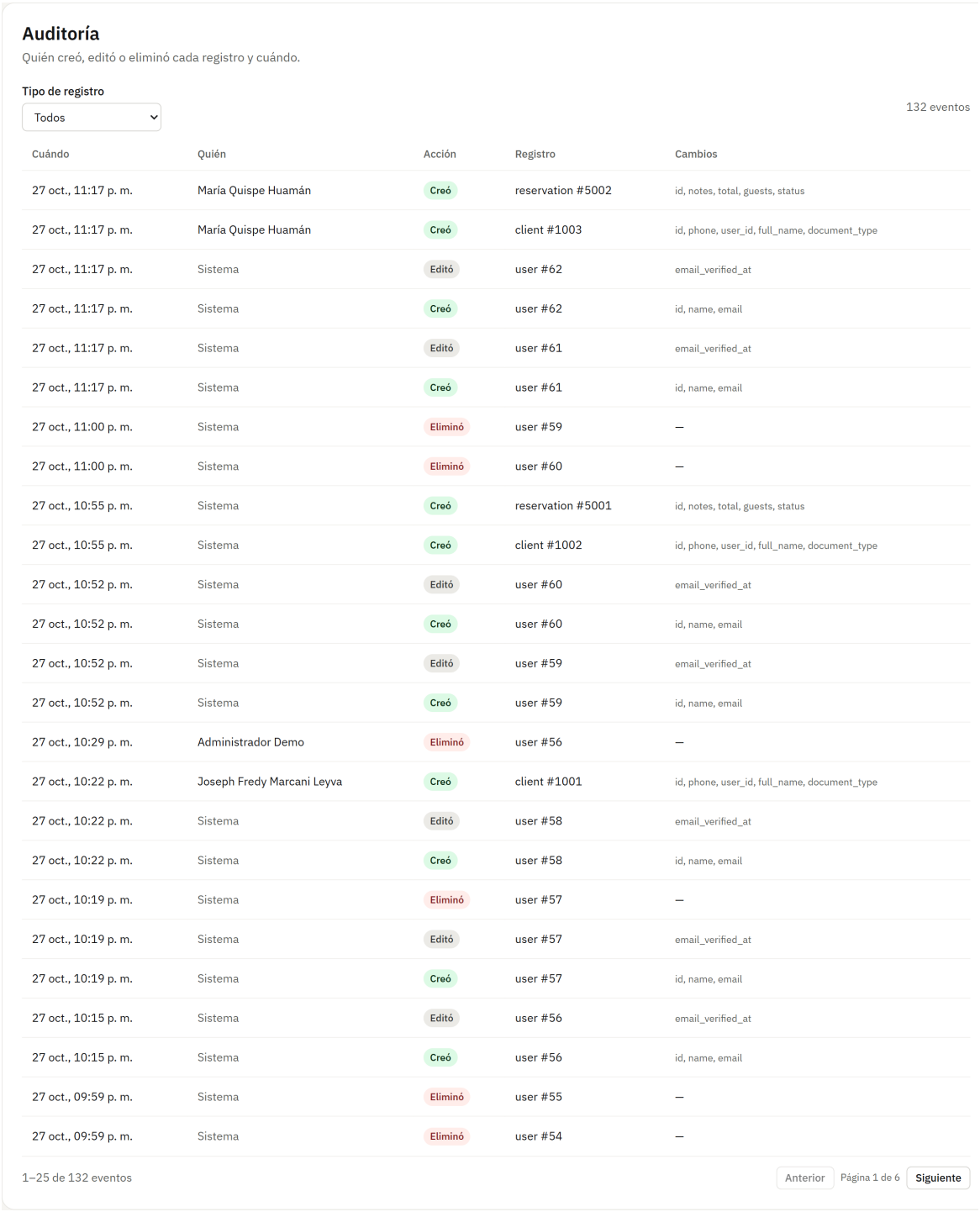


Figura 38. Auditoría

Facturación

Módulo previsto que todavía no está construido: muestra «En construcción» en vez de un error.



Figura 39. Facturación

Vista del programador

El Administrador también ve el diagnóstico de la API (detalle en la sección 4.4).

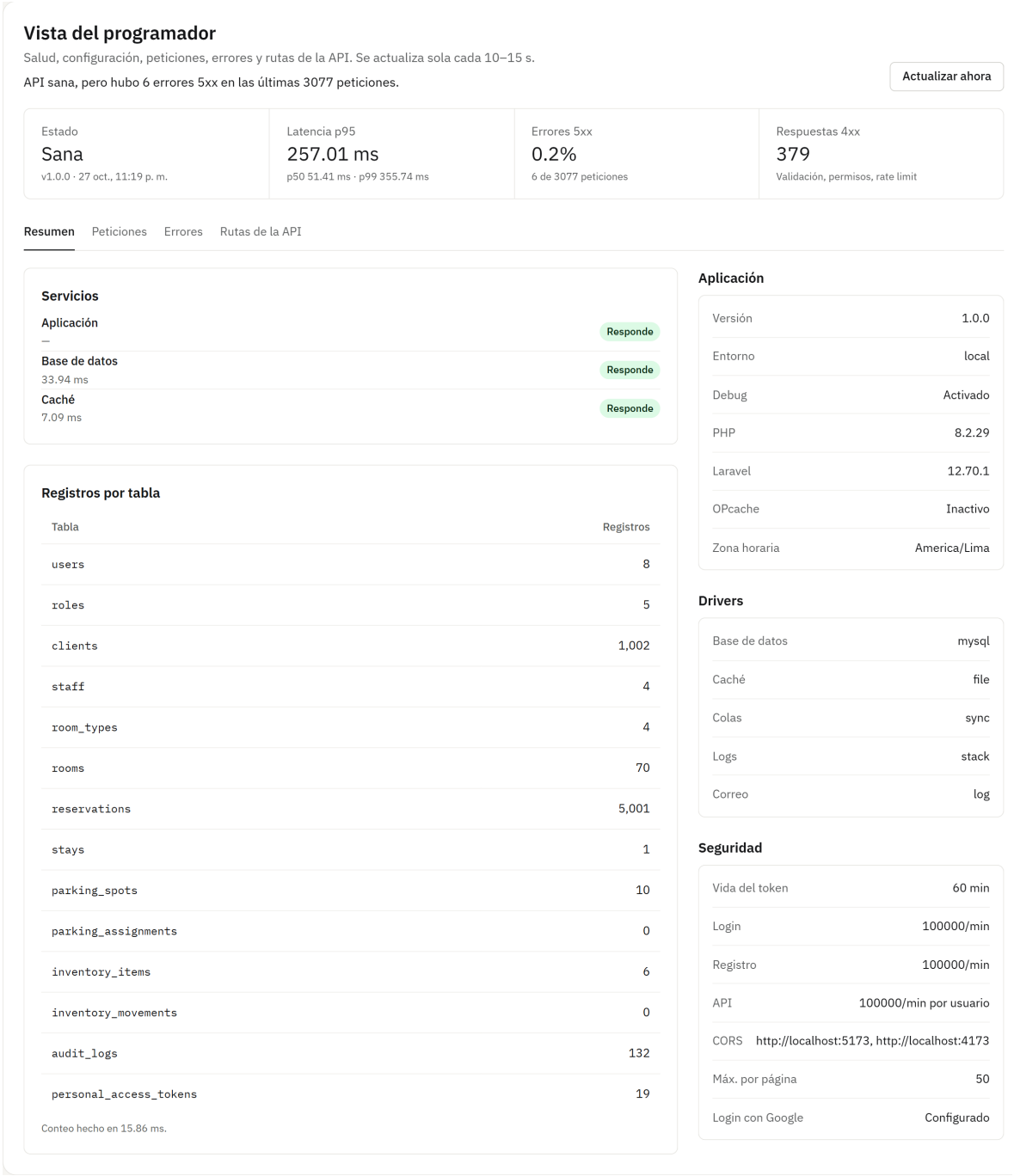


Figura 40. Vista del programador

4.4 Programador

Da soporte técnico: revisa el estado de la API sin entrar al servidor y registra cuentas del personal cuando hace falta.

Menú del Programador

Tiene dos opciones: Registrar personal y la Vista del programador.

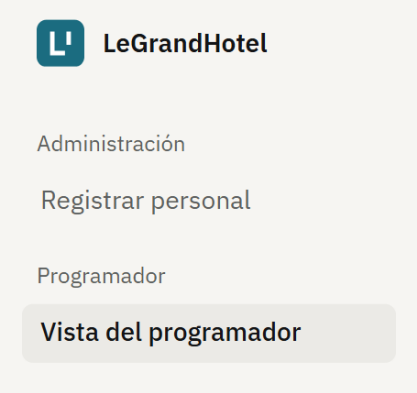


Figura 41. Menú del Programador.

Registrar personal

Igual que el Administrador, crea cuentas de Recepción, Almacén y Administración, pero no de Programador.

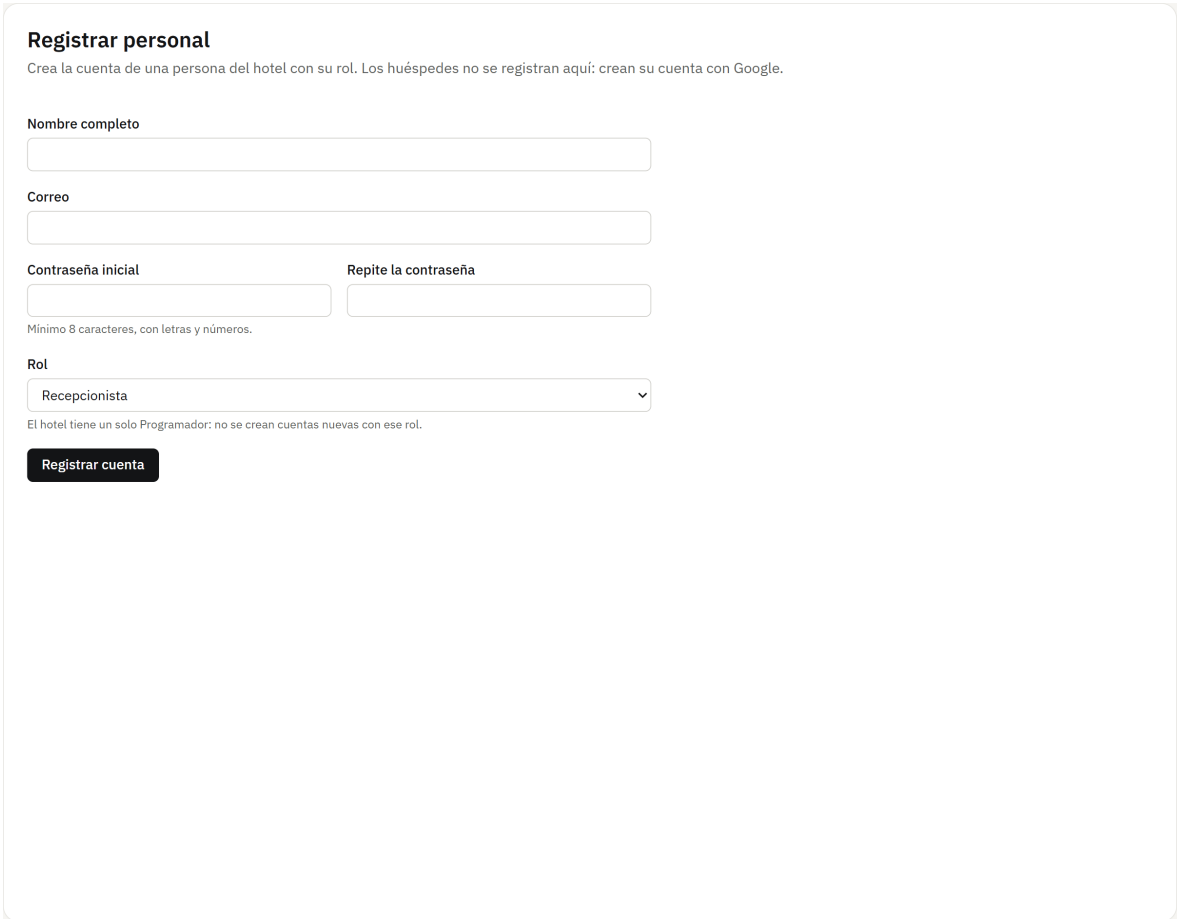


Figura 42. Registrar personal

Vista del programador · Resumen

Estado de la API, latencias p50, p95 y p99, porcentaje de errores 5xx y respuestas 4xx, y la configuración sin mostrar secretos. Se actualiza sola cada 10 a 15 segundos.

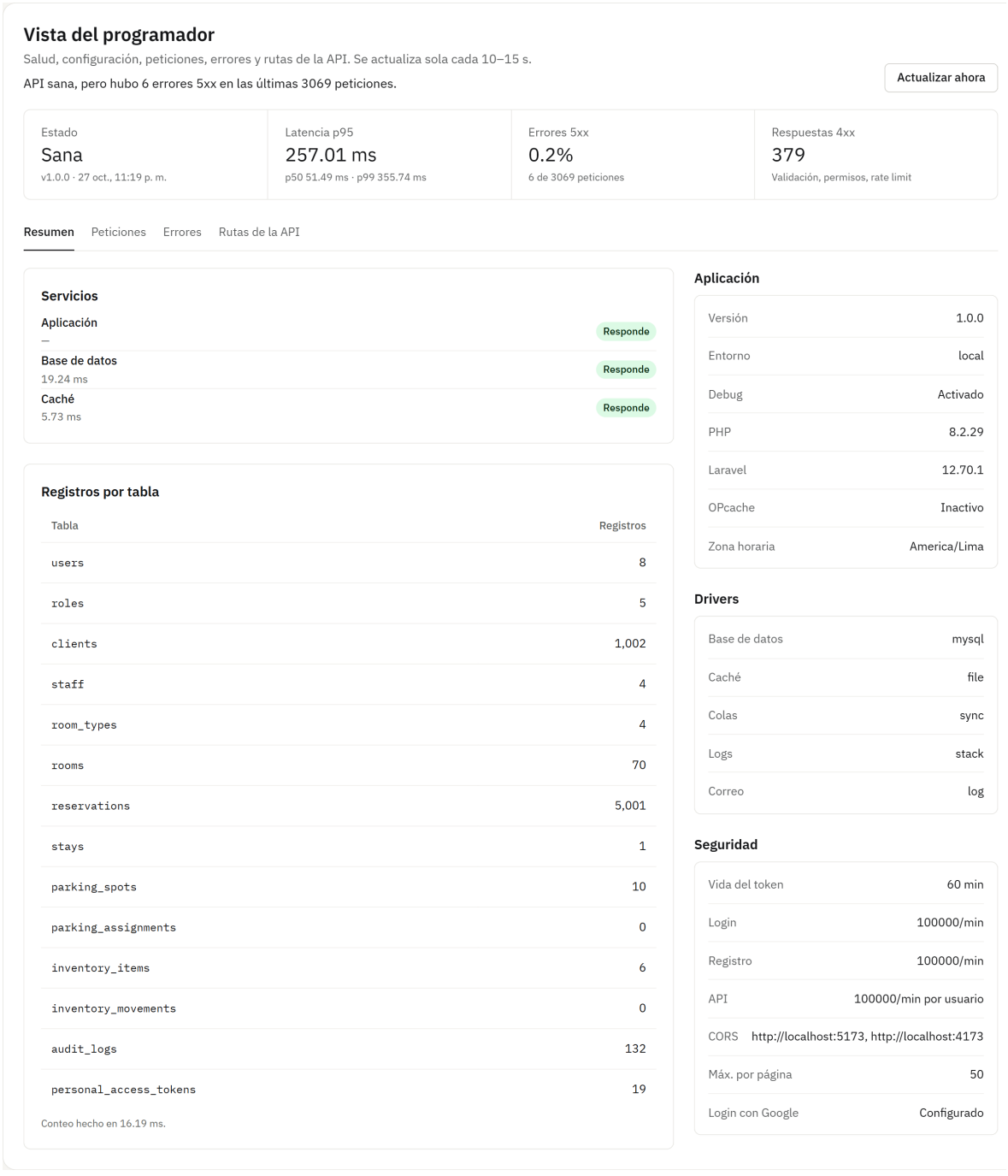
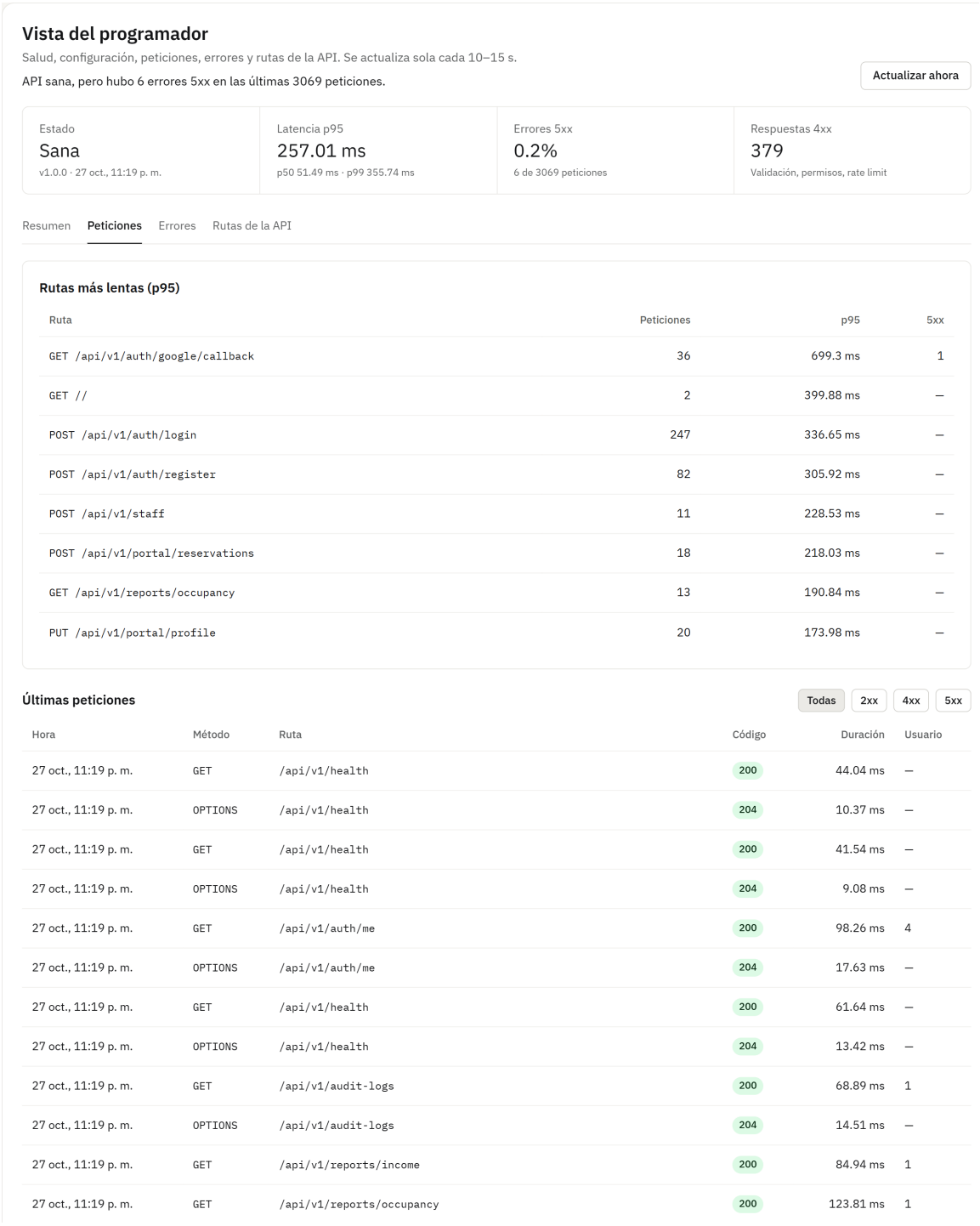


Figura 43. Vista del programador · Resumen

Vista del programador · Peticiones

Las rutas más lentas por p95 y las últimas peticiones con su código y su duración (se muestra la parte superior).



Vista del programador

Salud, configuración, peticiones, errores y rutas de la API. Se actualiza sola cada 10–15 s.

API sana, pero hubo 6 errores 5xx en las últimas 3069 peticiones.

Actualizar ahora

Estado	Latencia p95	Errores 5xx	Respuestas 4xx
Sana	257.01 ms	0.2%	379
v1.0.0 · 27 oct., 11:19 p. m.	p50 51.49 ms · p99 355.74 ms	6 de 3069 peticiones	Validación, permisos, rate limit

Resumen | Peticiones | Errores | Rutas de la API

Rutas más lentas (p95)

Ruta	Peticiones	p95	5xx
GET /api/v1/auth/google/callback	36	699.3 ms	1
GET //	2	399.88 ms	—
POST /api/v1/auth/login	247	336.65 ms	—
POST /api/v1/auth/register	82	305.92 ms	—
POST /api/v1/staff	11	228.53 ms	—
POST /api/v1/portal/reservations	18	218.03 ms	—
GET /api/v1/reports/occupancy	13	190.84 ms	—
PUT /api/v1/portal/profile	20	173.98 ms	—

Últimas peticiones

Todas | 2xx | 4xx | 5xx

Hora	Método	Ruta	Código	Duración	Usuario
27 oct., 11:19 p. m.	GET	/api/v1/health	200	44.04 ms	—
27 oct., 11:19 p. m.	OPTIONS	/api/v1/health	204	10.37 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/health	200	41.54 ms	—
27 oct., 11:19 p. m.	OPTIONS	/api/v1/health	204	9.08 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/auth/me	200	98.26 ms	4
27 oct., 11:19 p. m.	OPTIONS	/api/v1/auth/me	204	17.63 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/health	200	61.64 ms	—
27 oct., 11:19 p. m.	OPTIONS	/api/v1/health	204	13.42 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/audit-logs	200	68.89 ms	1
27 oct., 11:19 p. m.	OPTIONS	/api/v1/audit-logs	204	14.51 ms	—
27 oct., 11:19 p. m.	GET	/api/v1/reports/income	200	84.94 ms	1
27 oct., 11:19 p. m.	GET	/api/v1/reports/occupancy	200	123.81 ms	1

Figura 44. Vista del programador · Peticiones

Vista del programador · Errores

Errores del servidor, buscables por el código de incidente que el usuario ve en la pantalla 500. Aparece el incidente real JJA05WR4CG de la sección 6.

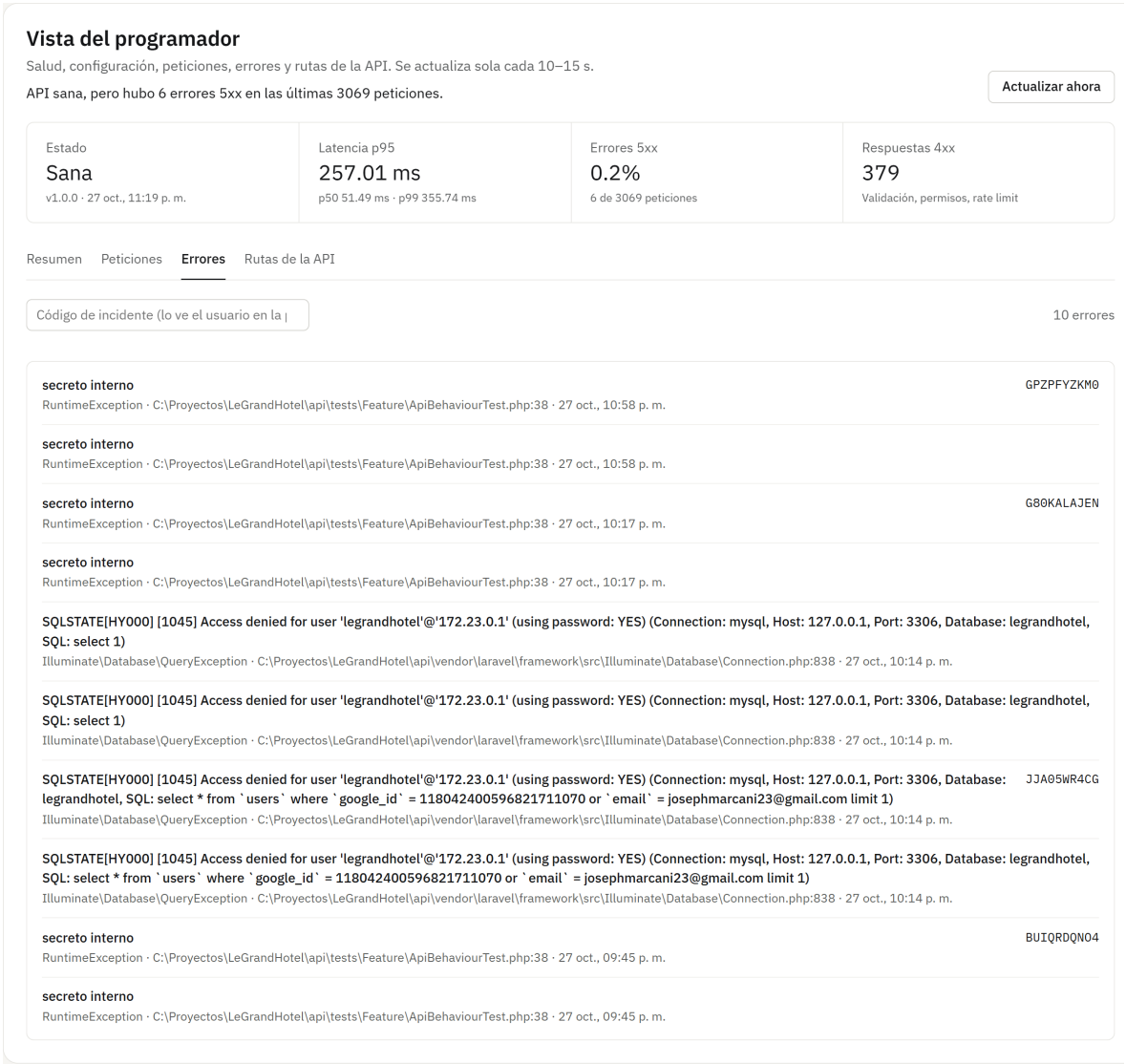
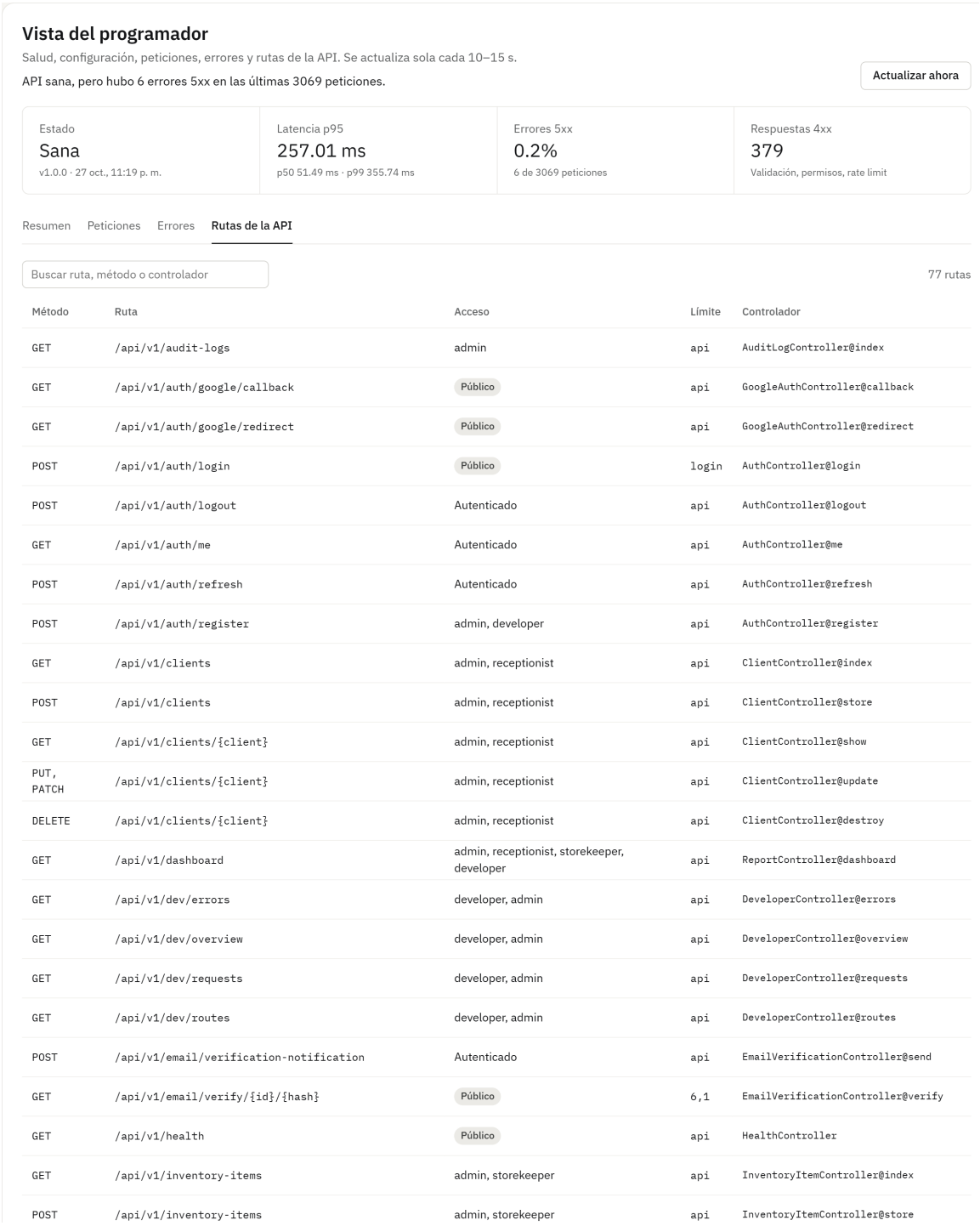


Figura 45. Vista del programador · Errores

Vista del programador · Rutas de la API

Las 77 rutas con su método, si piden sesión, los roles que pueden usarlas y su límite de peticiones (se muestra la parte superior).



Vista del programador

Salud, configuración, peticiones, errores y rutas de la API. Se actualiza sola cada 10–15 s.

API sana, pero hubo 6 errores 5xx en las últimas 3069 peticiones.

Actualizar ahora

Estado	Latencia p95	Errores 5xx	Respuestas 4xx
Sana	257.01 ms	0.2%	379
v1.0.0 · 27 oct., 11:19 p. m.	p50 51.49 ms · p99 355.74 ms	6 de 3069 peticiones	Validación, permisos, rate limit

Resumen | Peticiones | Errores | Rutas de la API

Buscar ruta, método o controlador

77 rutas

Método	Ruta	Acceso	Límite	Controlador
GET	/api/v1/audit-logs	admin	api	AuditLogController@index
GET	/api/v1/auth/google/callback	Público	api	GoogleAuthController@callback
GET	/api/v1/auth/google/redirect	Público	api	GoogleAuthController@redirect
POST	/api/v1/auth/login	Público	login	AuthController@login
POST	/api/v1/auth/logout	Autenticado	api	AuthController@logout
GET	/api/v1/auth/me	Autenticado	api	AuthController@me
POST	/api/v1/auth/refresh	Autenticado	api	AuthController@refresh
POST	/api/v1/auth/register	admin, developer	api	AuthController@register
GET	/api/v1/clients	admin, receptionist	api	ClientController@index
POST	/api/v1/clients	admin, receptionist	api	ClientController@store
GET	/api/v1/clients/{client}	admin, receptionist	api	ClientController@show
PUT, PATCH	/api/v1/clients/{client}	admin, receptionist	api	ClientController@update
DELETE	/api/v1/clients/{client}	admin, receptionist	api	ClientController@destroy
GET	/api/v1/dashboard	admin, receptionist, storekeeper, developer	api	ReportController@dashboard
GET	/api/v1/dev/errors	developer, admin	api	DeveloperController@errors
GET	/api/v1/dev/overview	developer, admin	api	DeveloperController@overview
GET	/api/v1/dev/requests	developer, admin	api	DeveloperController@requests
GET	/api/v1/dev/routes	developer, admin	api	DeveloperController@routes
POST	/api/v1/email/verification-notification	Autenticado	api	EmailVerificationController@send
GET	/api/v1/email/verify/{id}/{hash}	Público	6,1	EmailVerificationController@verify
GET	/api/v1/health	Público	api	HealthController
GET	/api/v1/inventory-items	admin, storekeeper	api	InventoryItemController@index
POST	/api/v1/inventory-items	admin, storekeeper	api	InventoryItemController@store

Figura 46. Vista del programador · Rutas de la API

4.5 Cliente

Es el huésped con cuenta propia: entra con Google, completa su DNI y teléfono una vez y reserva en línea. No ve nada del panel del personal.

Completar registro (antes del menú)

La primera vez que entra con Google aparece esta pantalla, todavía sin menú. El nombre y el correo vienen de Google; el huésped escribe su DNI (8 dígitos) y su teléfono (9 dígitos). Si su DNI ya estaba registrado en recepción, se vincula esa ficha.

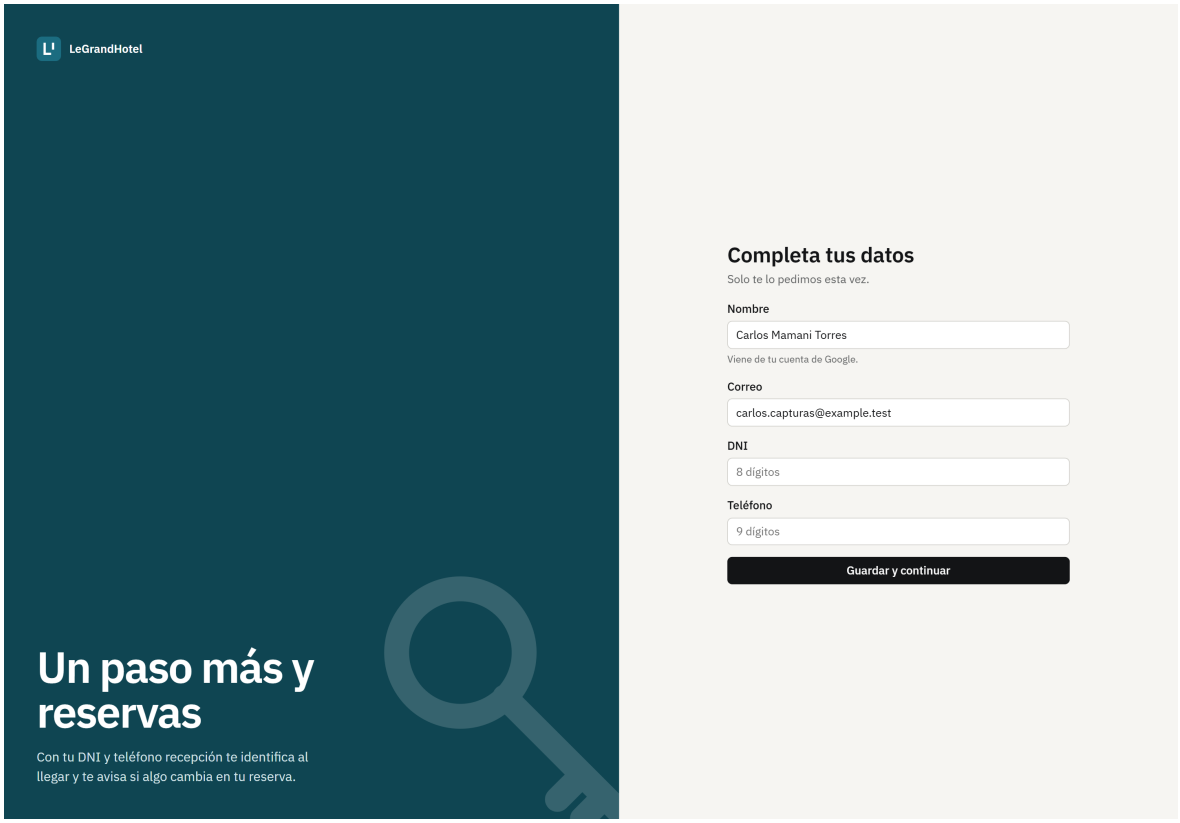


Figura 47. Completar registro (antes del menú)

Menú del Cliente

Su menú «Mi cuenta» tiene dos opciones: Reservar y Mis reservas.



Figura 48. Menú del Cliente.

Reservar

Elige llegada, salida y personas; ve cada tipo de habitación con lo que incluye, cuántas quedan libres y el total de su estadía. Los tipos que no alcanzan para esa cantidad de personas no aparecen.

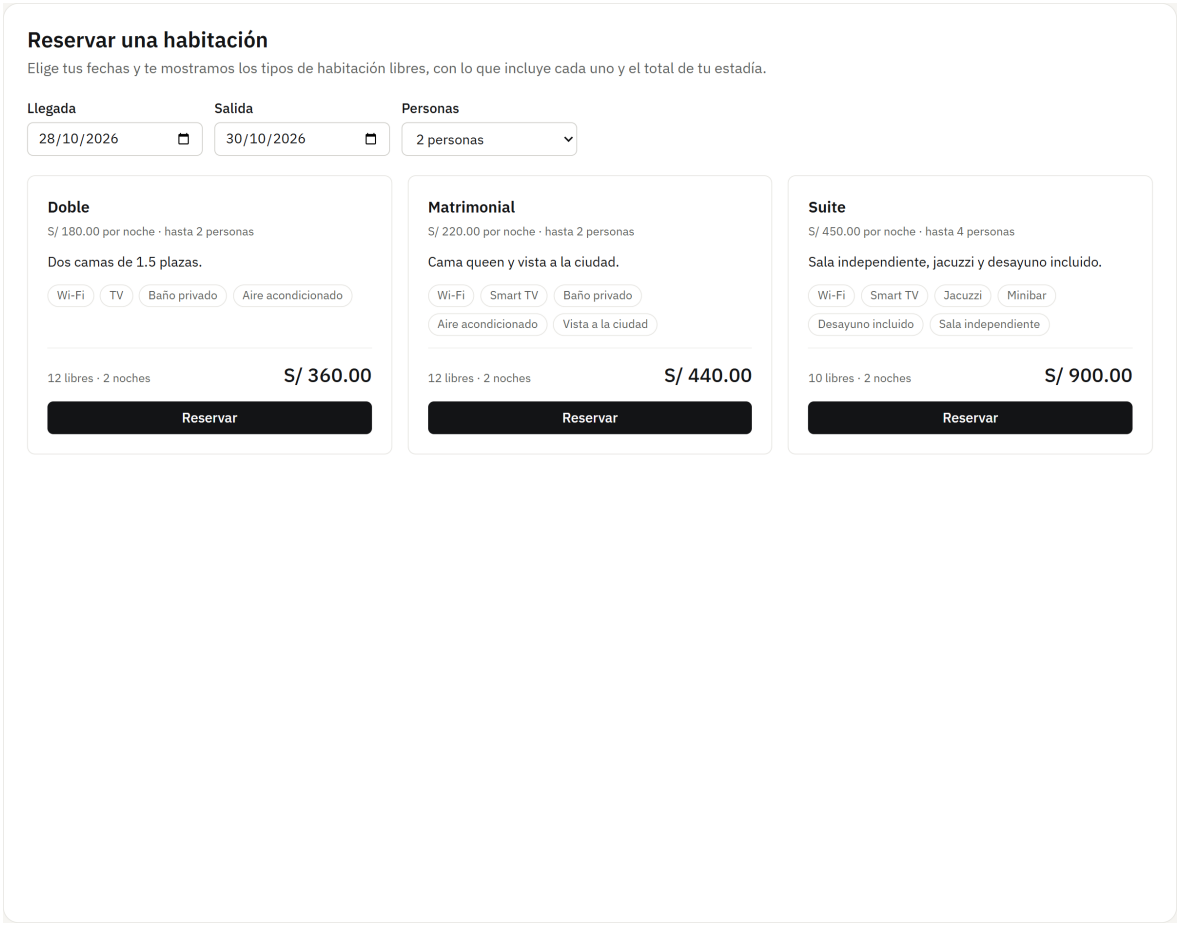


Figura 49. Reservar

Ventana emergente: Confirmar reserva

Resumen con el tipo, las fechas, las noches y el total antes de confirmar. Al confirmar, el hotel le asigna una habitación libre de ese tipo.



Figura 50. Ventana emergente: Confirmar reserva

Mis reservas

Sus reservas con habitación, fechas, noches, total y estado. Puede cancelar una reserva confirmada mientras falte al menos un día para la llegada.

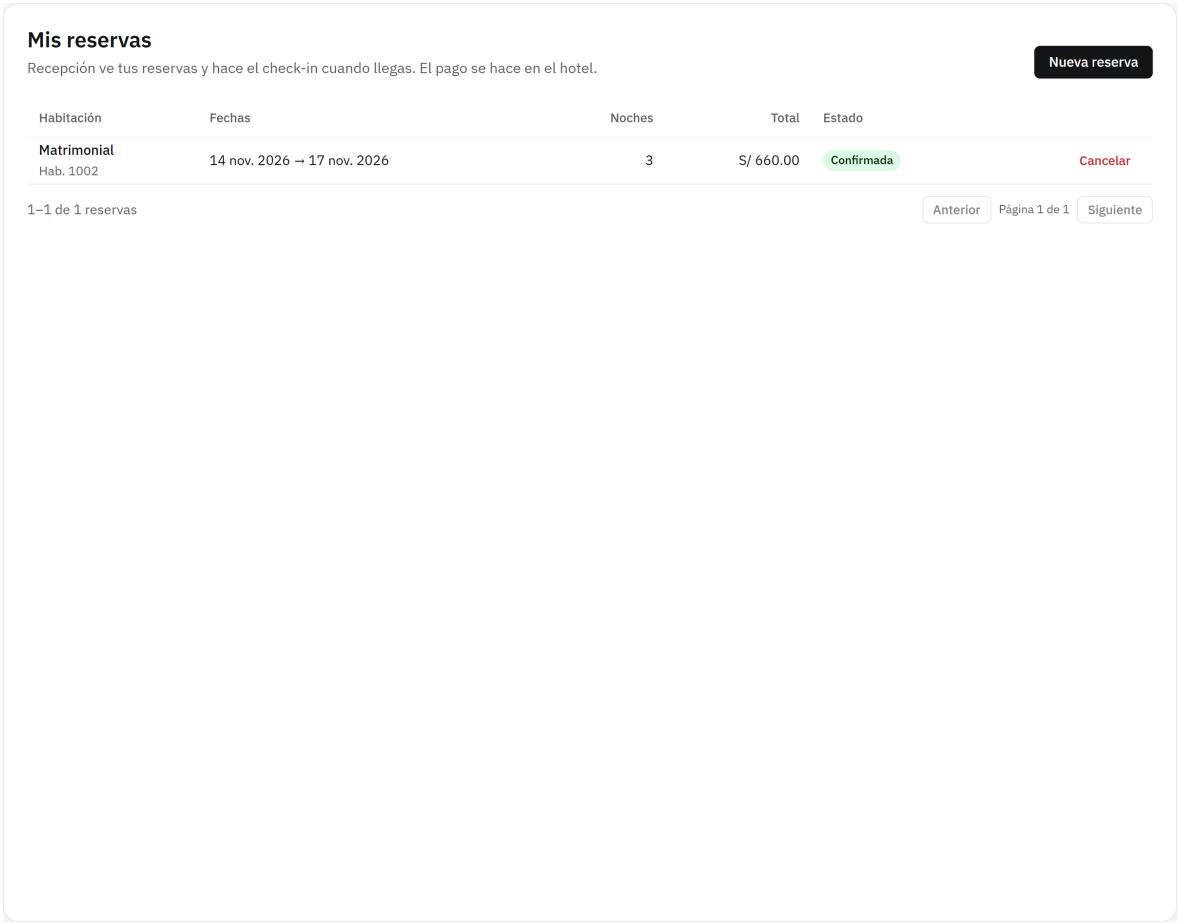


Figura 51. Mis reservas

5. Pantallas de error

Cada respuesta de error de la API tiene su pantalla, con el mismo estilo de la página, un mensaje claro y una salida (volver al inicio, recargar o iniciar sesión). Las decide un solo interceptor en web/src/lib/api.ts, así se comportan igual en todas las pestañas.

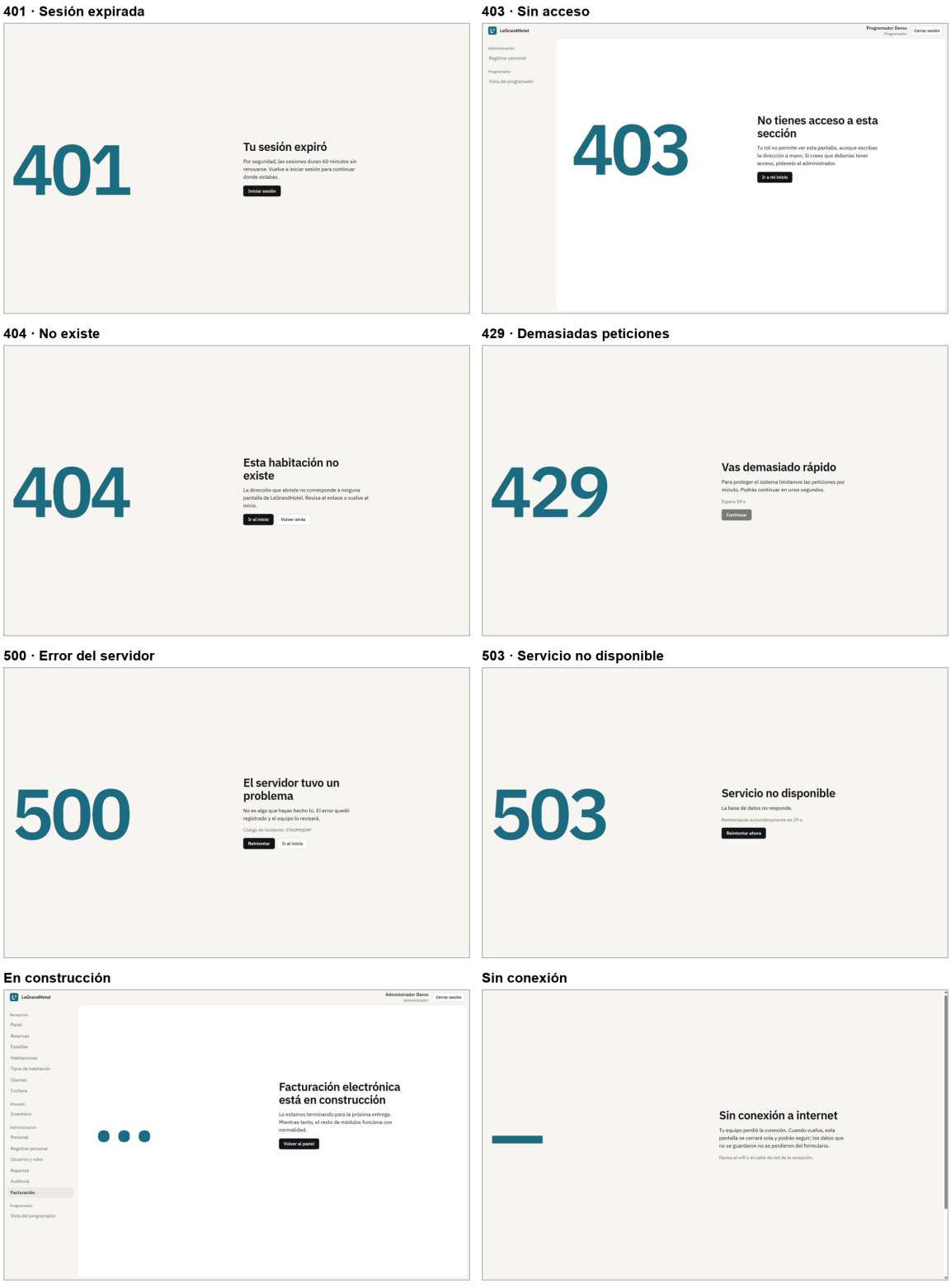


Figura 52. Pantallas 401, 403, 404, 429, 500, 503, en construcción y sin conexión.

Pantalla	Cuándo aparece
401 · Sesión expirada	El token venció (60 minutos) o se cerró la sesión en otro lugar
403 · Sin acceso	El rol no tiene permiso para esa pestaña o esa acción
404 · No encontrada	La dirección no existe o el registro fue eliminado
429 · Demasiadas peticiones	Se superó el límite (por ejemplo 5 logins por minuto); muestra cuánto esperar

Pantalla	Cuándo aparece
500 · Error del servidor	Algo falló; muestra el código de incidente para buscarlo en la Vista del programador
503 · No disponible	La API no responde; reintenta sola cada 30 segundos
En construcción	Módulo previsto pero aún no construido (Facturación)
Sin conexión	El equipo perdió internet; vuelve sola al reconectarse

6. Registro de incidentes

Toda caída o falla, aunque sea en el entorno local o en pruebas, se anota el mismo día en docs/incidentes.md con su causa y su solución. El registro tiene ahora 8 incidentes; 6 son del 27 de octubre, al repetir las pruebas de carga, configurar el inicio con Google y revisar la Vista del programador.

Fecha	Qué falló	Causa	Solución	Tiempo
2026-10-26	El login devolvía 500 INTERNAL_ERROR con código de incidente XG1R7UASWT	php artisan serve lanza procesos PHP hijos sin la extensión pdo_sqlite, cargada solo con -d en el proceso padre ("could not find driver")	Se arranca el servidor local con php -d extension=pdo_sqlite -S localhost:8000 -t api/public api/public/index.php (.claude/launch.json). El error salió como JSON sin trazas, como exige el plan	~2 min
2026-10-26	El frontend mostró la pantalla 503 "No se pudo contactar con el servidor"	Durante el cambio anterior el servidor PHP apuntaba a un server.php que buscaba index.php en la carpeta equivocada	Se usa api/public/index.php como router del servidor. La pantalla 503 reintentó sola cada 30 s	~1 min
2026-10-27	Primera serie de k6: 100% de peticiones fallidas	El servidor PHP escuchaba solo en IPv6 (::1) y k6 resolvía localhost a IPv4 (127.0.0.1)	BASE_URL=http://[::1]:8000/api/v1; se borraron los resultados inválidos y se repitieron las 5 corridas	~5 min
2026-10-27	El primer inicio con Google devolvió 500 INTERNAL_ERROR (incidente JJA05WR4CG) y /health marcó la base «down»	Al editar api/.env para agregar GOOGLE_CLIENT_ID y GOOGLE_CLIENT_SECRET se borró un carácter de DB_PASSWORD: MySQL respondió «Access denied for user 'legrandhotel'»	Se restauró DB_PASSWORD igual a MYSQL_PASSWORD de docker-compose.yml; /health volvió a «ok». El código de incidente permitió hallar la causa en storage/logs en segundos	~1 min
2026-10-27	Después de entrar con Google el navegador mostró ERR_INVALID_REDIRECT	FRONTEND_URL lista dos dominios separados por coma (para CORS) y el callback de Google y la verificación de correo lo usaban completo como dirección de regreso	Nuevo App\Support\Frontend::url() que redirige al primer dominio; prueba que lo cubre en ClientPortalTest	~3 min
2026-10-27	k6 run se detenía en setup() con «La API no está sana antes de la prueba: 503»	El contenedor nginx de docker compose ocupaba el puerto 8000; k6 hablaba con ese Nginx (sin PHP detrás) y no con la API	docker compose down y encender solo MySQL (docker compose up -d mysql); el tutorial de k6 lo explica en el paso 2	~10 min
2026-10-27	Al volver de Google la página quedaba en blanco con #token=… en la dirección	La aplicación no llegó a arrancar en esa pestaña de Brave (la API nunca recibió /auth/me), probablemente por sus escudos (Shields); en Chrome el mismo flujo funcionó	Usar Chrome o desactivar Shields para localhost; se comprobó el flujo completo con una cuenta de prueba	~15 min

Fecha	Qué falló	Causa	Solución	Tiempo
2026-10-27	La pestaña «Errores» mostró la pantalla 500 con código WEB-MVRL1WBX	Laravel guarda algunas excepciones en el log como objeto (class, message, file) y la pantalla intentaba mostrarlas como texto (error de React)	RequestLogReader devuelve siempre la clase como texto; prueba nueva en DeveloperViewTest	~5 min

Cómo ayudó el código de incidente: cuando la API falla, responde 500 con un código (por ejemplo JJA05WR4CG) que el usuario ve en pantalla. El mismo código queda en el registro JSON de la API, y la Vista del programador lo encuentra en segundos (figura de la sección 4.4). Así se halló que el problema del primer inicio con Google era la contraseña de la base de datos, no Google.

Registro de la API (storage/logs/laravel.json), resumido

```
{"message": "SQLSTATE[HY000] [1045] Access denied for user 'legrandhotel'…",
 "context": {"incident_id": "JJA05WR4CG", "exception":
"Illuminate\\Database\\QueryException"},
 "level_name": "ERROR", "datetime": "2026-10-27T22:14:14-05:00"}
```

7. Pruebas de rendimiento y estrés con k6

k6 simula muchos «usuarios virtuales» (VU) usando la API a la vez y mide cuánto tarda cada respuesta. Cada guion tiene metas (thresholds): si no se cumplen, k6 marca la prueba con ✗ y termina con código 99. Las pruebas de carga se hicieron con clientes, como si muchos huéspedes usaran el portal de reservas al mismo tiempo. Las corridas se hicieron en la laptop del equipo, con la API en el servidor integrado de PHP (php -S, que atiende una petición a la vez) y la base MySQL con 70 habitaciones y 5000 reservas de demostración.

Prueba	Qué hace	Para qué
1. Humo (smoke)	5 VU durante 30 s piden /health y una ruta protegida sin token	Confirmar que la API, la base y la caché funcionan antes de cargar
2. Límite de login	6 intentos de login con contraseña incorrecta	Demostrar la protección contra fuerza bruta (429)
3 a 6. Escalones de clientes	Clientes del portal (60 % buscan habitaciones libres, 25 % ven sus reservas y 15 % reservan) con 5, 10, 25 y 50 VU, 1 minuto cada uno	Ver cuántos clientes a la vez aguanta el portal con buena velocidad y si aparecen errores

Cómo se simulan los clientes. Los clientes solo entran con Google y k6 no puede iniciar sesión en Google. Por eso LoadTestClientsSeeder crea 50 clientes de prueba con su DNI y teléfono y les abre la sesión (un token por cliente), y el guion tests/k6/portal.js usa esas sesiones. Al terminar, php artisan k6:limpiar borra esos clientes y las reservas que hicieron.

tests/k6/portal.js (lo que hace cada cliente en cada vuelta)

```
if (dice < 0.6) {          // 60 %: busca habitaciones libres para unas fechas
  http.get(`${BASE_URL}/portal/room-types?from=${from}&to=${to}&guests=2`, auth(token))
} else if (dice < 0.85) {  // 25 %: revisa sus reservas
  http.get(`${BASE_URL}/portal/reservations`, auth(token))
} else {                   // 15 %: busca y reserva un tipo con habitaciones libres
  http.post(`${BASE_URL}/portal/reservations`, JSON.stringify({ room_type_id, check_in,
check_out, guests: 1 }), {
    ...auth(token), responseCallback: http.expectedStatuses(201, 409) })  // 409 = otro
cliente la tomó antes
}
sleep(1 + Math.random() * 2)   // pausa de 1 a 3 s, como una persona
```

7.1 Prueba de humo

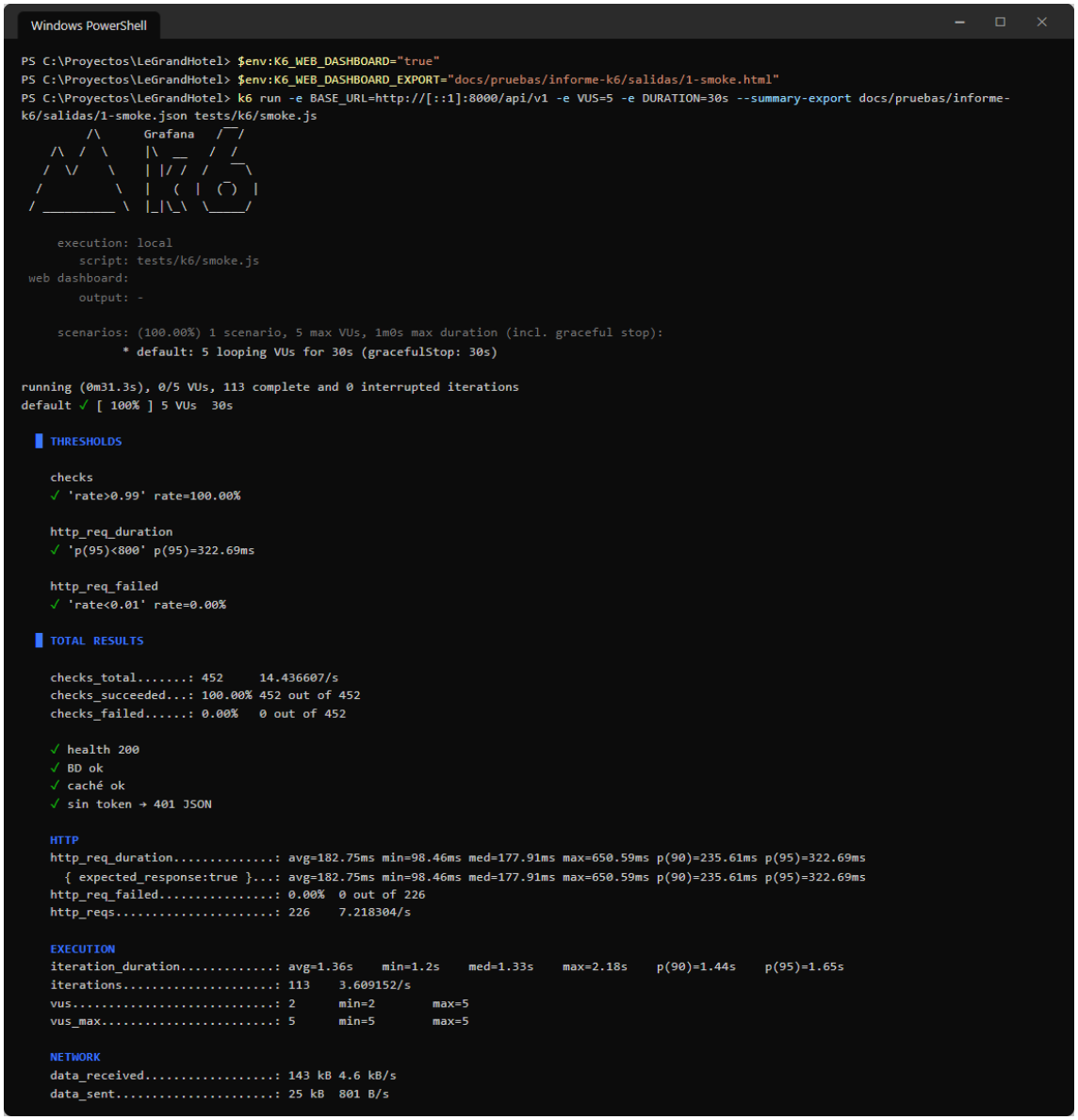


Figura 53. Comando de la prueba de humo y la respuesta de k6.

Interpretación: las tres metas salen con ✓. El 95 % de las respuestas tardó menos de 323 ms (meta: 800 ms), no hubo peticiones fallidas (0.00 %) y los 452 checks pasaron: la API, la base de datos y la caché responden, y una ruta protegida sin token devuelve 401 en JSON.

7.2 Límite de intentos de login

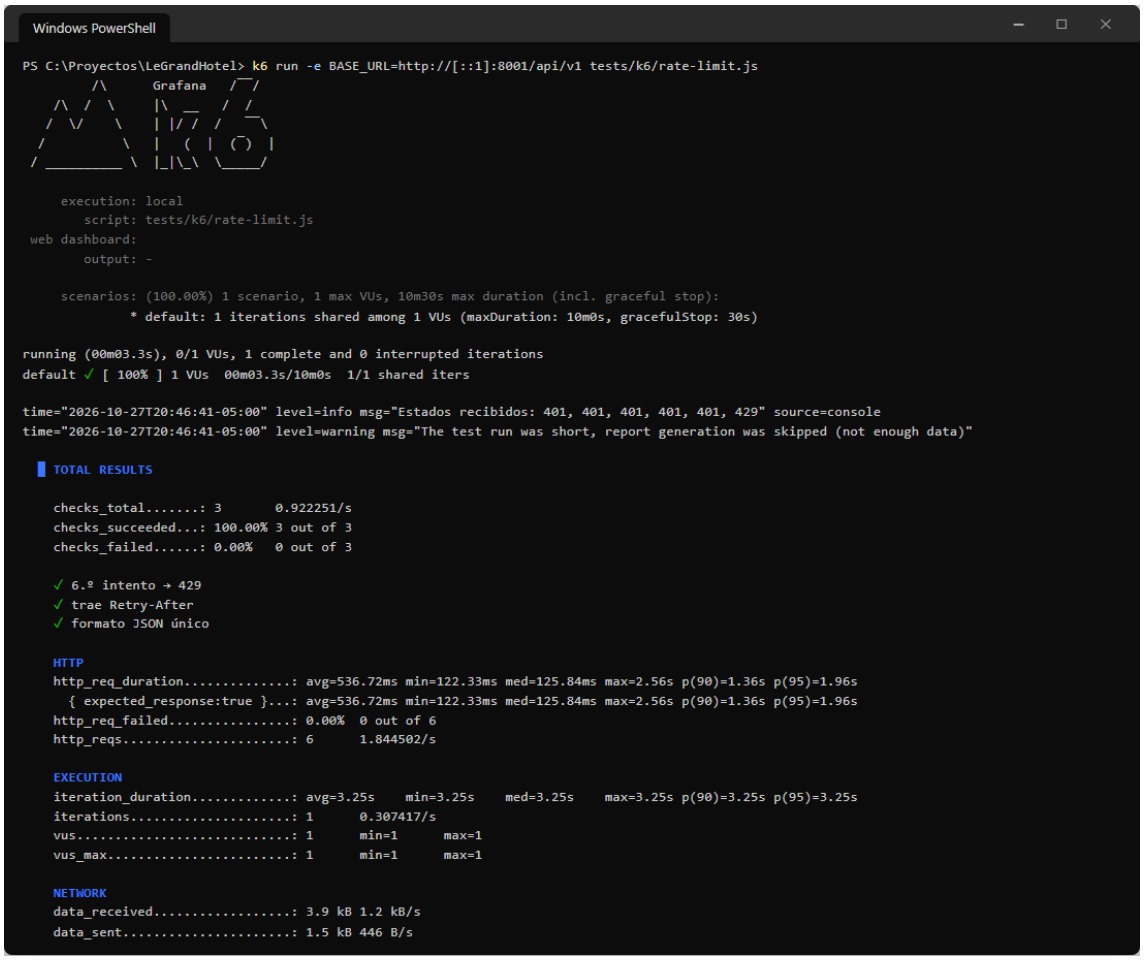


Figura 54. Prueba del límite de login: el sexto intento recibe 429.

Interpretación: «Estados recibidos: 401, 401, 401, 401, 401, 429». Los cinco primeros intentos responden 401 (contraseña incorrecta) y el sexto 429 con la cabecera Retry-After y el código TOO_MANY_REQUESTS. Nadie puede probar contraseñas sin límite.

7.3 Escalones de clientes (5, 10, 25 y 50 a la vez)

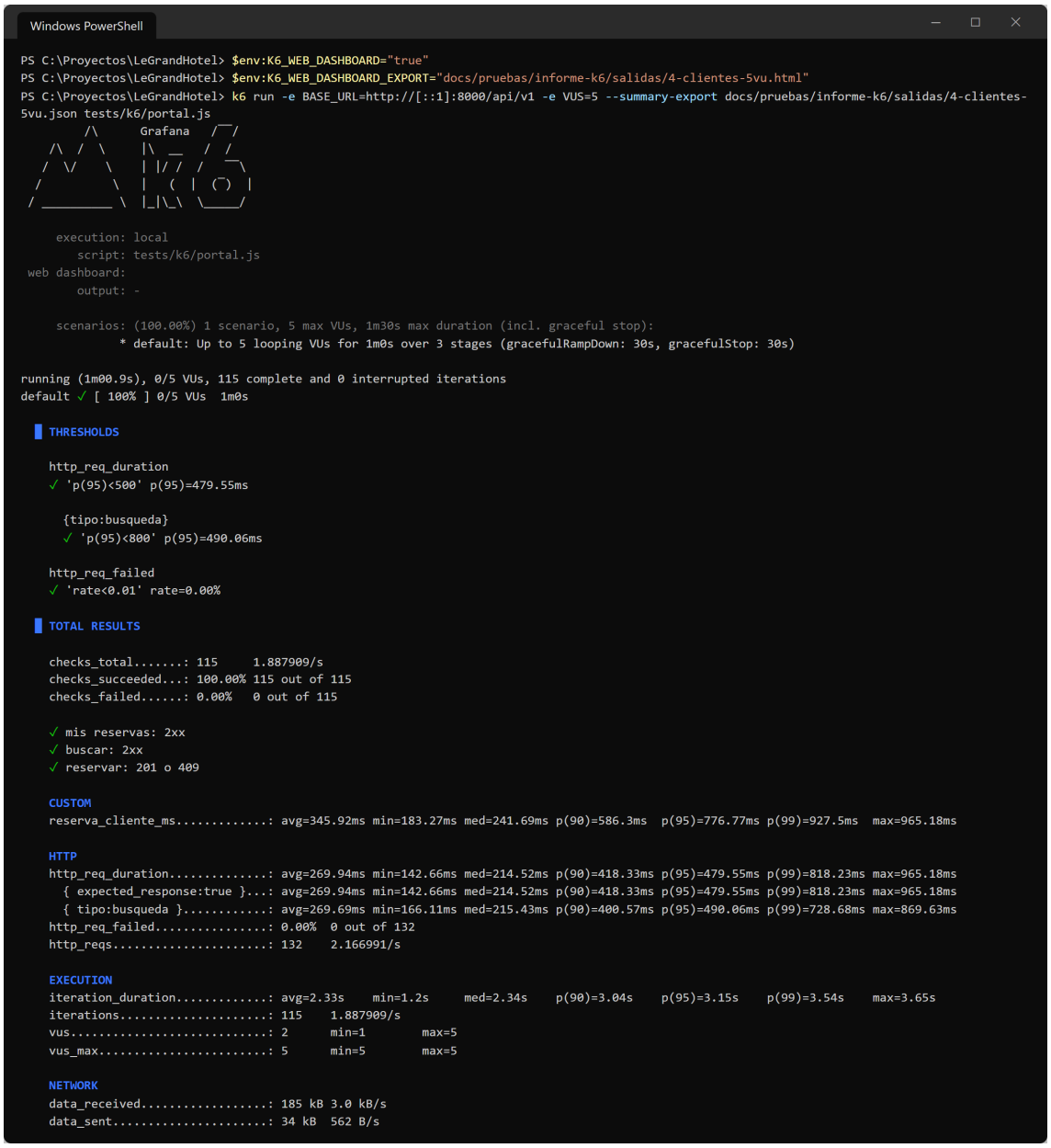


Figura 55. Escalón de 5 clientes: comando y respuesta.

Interpretación: con 5 clientes a la vez la prueba se aprueba completa: todas las metas salen con ✓. El 95 % de las respuestas tardó menos de 480 ms (meta: 500 ms), las búsquedas 490 ms (meta: 800 ms), no hubo errores y pasaron los 115 checks: buscar, ver sus reservas y reservar (201, o 409 si otro cliente tomó la última habitación). Crear una reserva tardó 777 ms en el p95.



Figura 56. Escalón de 50 clientes: comando y respuesta.

Interpretación: con 50 clientes a la vez siguen pasando todos los checks y no hay errores (0.00 %), pero las metas de velocidad salen con ✗: el p95 sube a 11,099 ms y crear una reserva a 11,223 ms. La línea final «thresholds … have been crossed» es k6 avisando que la prueba no se aprobó por lentitud, no por fallas.

Clientes	Peticiones	Pet./s	Mediana	p95	Reservar (p95)	Errores	Meta p95 < 500 ms
5	132	2.2	215 ms	480 ms	777 ms	0.00 %	✓
10	236	3.8	481 ms	1,750 ms	1,834 ms	0.00 %	✗
25	278	4.4	3,479 ms	4,369 ms	4,461 ms	0.00 %	✗
50	317	4.6	8,031 ms	11,099 ms	11,223 ms	0.00 %	✗

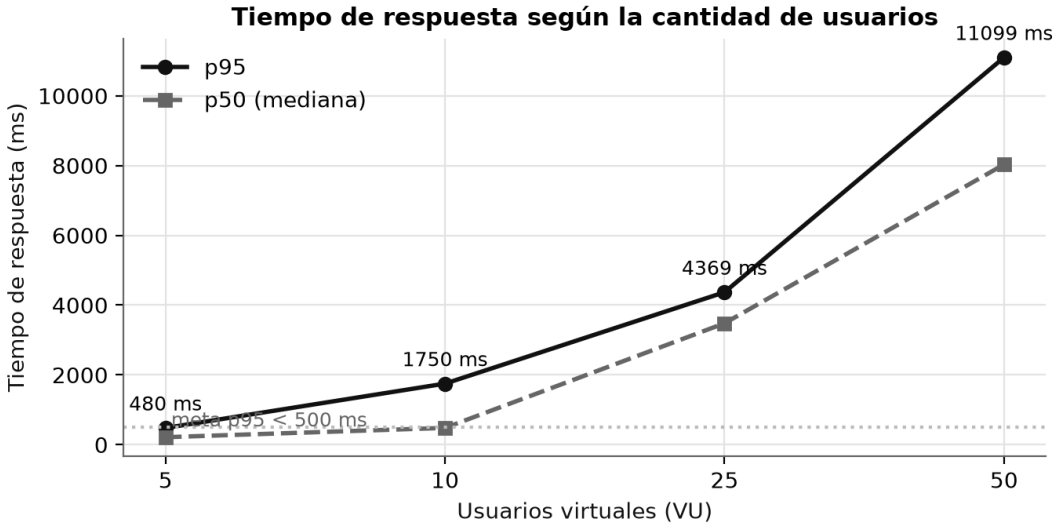


Figura 57. Tiempo de respuesta (mediana y p95) según la cantidad de clientes.

Interpretación del gráfico: el eje horizontal son los clientes a la vez y el vertical el tiempo de respuesta; la línea punteada es la meta de 500 ms. Con 5 clientes el p95 (480 ms) queda justo debajo de la meta. Desde 10 clientes las dos curvas suben casi en línea recta hasta 11,099 ms con 50. Que crezca en proporción a los clientes es la señal de una cola: cada cliente nuevo espera a los anteriores.

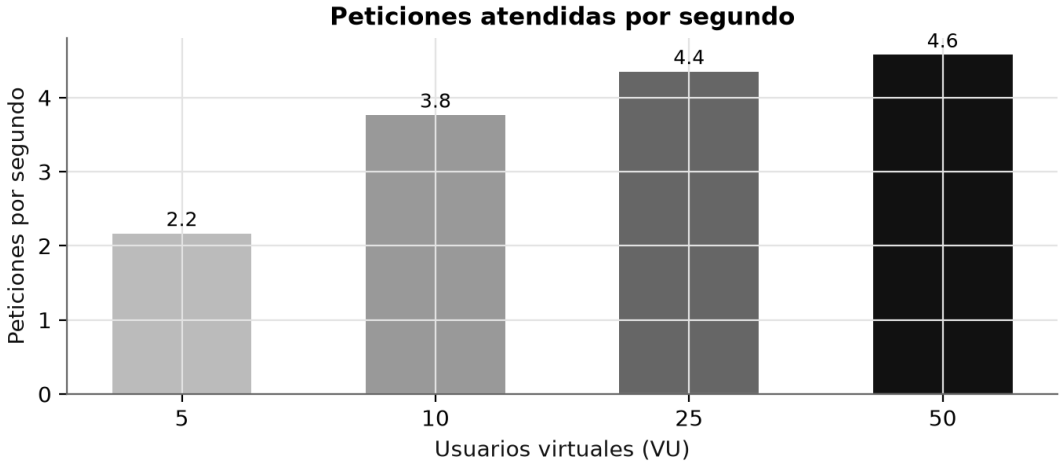


Figura 58. Peticiones atendidas por segundo en cada escalón de clientes.

Interpretación del gráfico: de 5 a 25 clientes las peticiones por segundo suben (de 2.2 a 4.4), porque todavía hay capacidad libre. De 25 a 50 casi no cambian (4.4 → 4.6): la API llegó a su techo de unas 4,6 peticiones por segundo y los clientes extra solo esperan más. Junto con el gráfico anterior confirma el cuello de botella.

7.4 Reporte HTML de k6 (escalón de 50 clientes)

k6 genera además un reporte con gráficos de la prueba (salidas/4-clientes-50vu.html). Sus tres gráficos se muestran por separado. El eje horizontal es la hora (de 07:51:03 a 07:51:53) y cada punto resume 10 segundos.

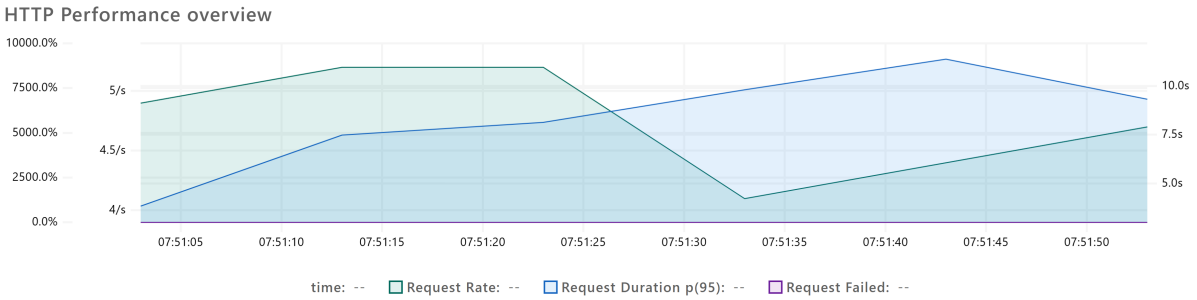


Figura 59. HTTP Performance overview: peticiones por segundo, p95 y peticiones fallidas (50 clientes).

Qué muestra: tres medidas a la vez, cada una con su propio eje. La línea verde es la cantidad de peticiones por segundo (eje de 4/s a 5/s); la azul, el p95 del tiempo de respuesta (eje derecho, en segundos); la morada, el porcentaje de peticiones fallidas (eje izquierdo).

Interpretación: las peticiones por segundo se mueven en una franja angosta, entre unas 4,1/s y 4,9/s, sin poder subir más: es el techo de la API. Mientras tanto el p95 crece de unos 4 s al inicio a unos 8 s a las 07:51:23, cuando ya están los 50 clientes, y llega a unos 11 s a las 07:51:43. La línea morada está pegada en 0 % todo el tiempo: ninguna de las 317 peticiones falló. Conclusión: con 50 clientes el portal se pone lento, pero no se cae ni pierde reservas.

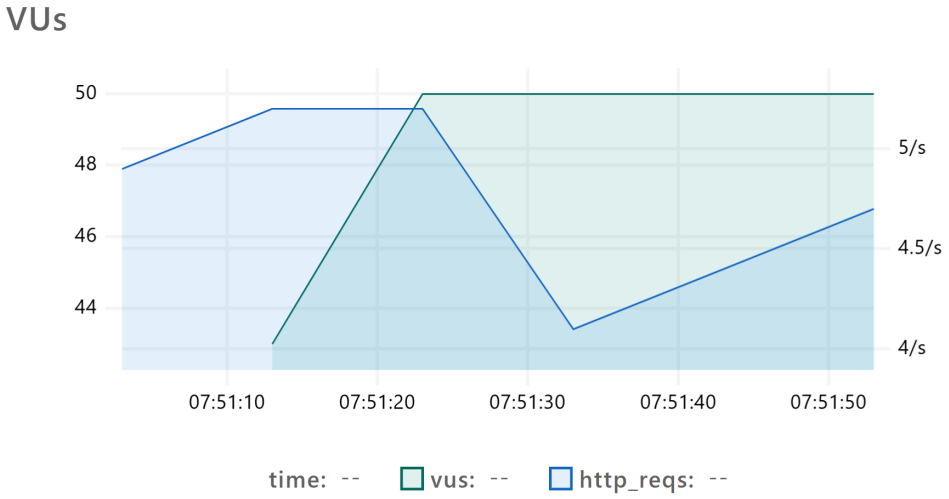


Figura 60. VUs: clientes activos y peticiones por segundo.

Qué muestra: la línea verde son los clientes activos (eje izquierdo); la azul, las peticiones por segundo (eje derecho). El eje de clientes empieza en 42 para que se vea el detalle.

Interpretación: los clientes pasan de 43 a 50 entre las 07:51:13 y las 07:51:23 y se quedan en 50 hasta el final. Al llegar a 50 las peticiones por segundo no suben: bajan de 4,9/s a 4,1/s a las 07:51:33 y luego se recuperan hasta 4,6/s. Más clientes no significa más trabajo atendido: la

API ya estaba en su techo y la cola crece. A diferencia del escenario del personal, aquí no hay inicios de sesión al comienzo, porque los clientes de prueba ya tienen su sesión abierta.

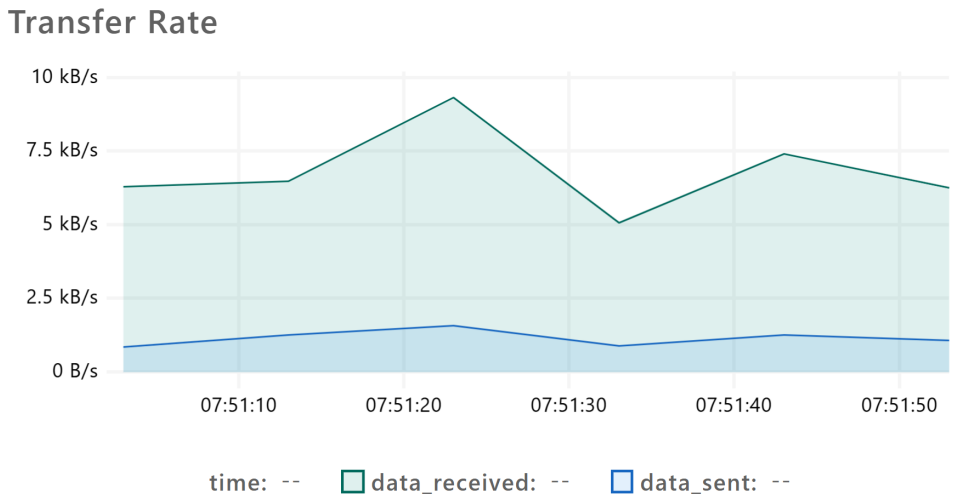


Figura 61. Transfer Rate: datos recibidos y enviados por segundo (50 clientes).

Qué muestra: cuántos datos recibió k6 de la API (data_received, verde) y cuántos le envió (data_sent, azul), en kilobytes por segundo.

Interpretación: lo recibido va de 5 a 9,3 kB/s, con un pico a las 07:51:23; lo enviado se mantiene entre 1 y 1,5 kB/s. Son cifras pequeñas porque las respuestas del portal son livianas (los tipos de habitación y las reservas de un solo cliente). En total fueron 483 kB recibidos y 83 kB enviados: la red no es el límite, la lentitud viene de la API que procesa una petición a la vez.

7.5 Conclusión de las pruebas

- Hasta 5 clientes a la vez el portal cumple todas las metas (p95 de 480 ms).
- Correcta y estable: en todas las corridas no hubo ni una petición fallida ni errores 500, y las reservas simultáneas sobre la misma habitación se resolvieron con 409, sin reservas dobles.
- Segura bajo carga: 401 sin sesión y 429 al pasar el límite de login, siempre en el formato JSON de la API.
- Desde 10 clientes se pone lenta en la laptop por el servidor de desarrollo de PHP, que atiende una petición a la vez (techo de unas 4,6 peticiones por segundo). La solución es correr la API con varios procesos PHP en paralelo y caché de código (OPcache) en el servidor definitivo, y repetir estas mismas pruebas allí, con más clientes, cuando se defina el servidor.

8. Diseño de la aplicación Android e iOS

LeGrandHotel tendrá una aplicación para teléfonos que usa la misma API y los mismos roles que la web, sin duplicar ninguna regla del negocio. Cubre lo que conviene hacer desde el

celular: el huésped reserva y revisa sus reservas, recepción atiende llegadas y salidas desde el lobby, y el almacén registra movimientos en el depósito.

Rol	**Qué hará en la app**
Cliente	Entrar con Google, completar DNI y teléfono, reservar por tipo de habitación, ver y cancelar sus reservas, recibir un recordatorio un día antes de llegar
Recepcionista	Pantalla «Hoy» con llegadas y salidas, check-in y check-out, reservas, estado de habitaciones y cochera
Almacenero	Ver el inventario con los insumos bajo el mínimo y registrar entradas y salidas
Administrador	El panel resumido (ocupación, ingresos y alertas); la gestión completa sigue en la web

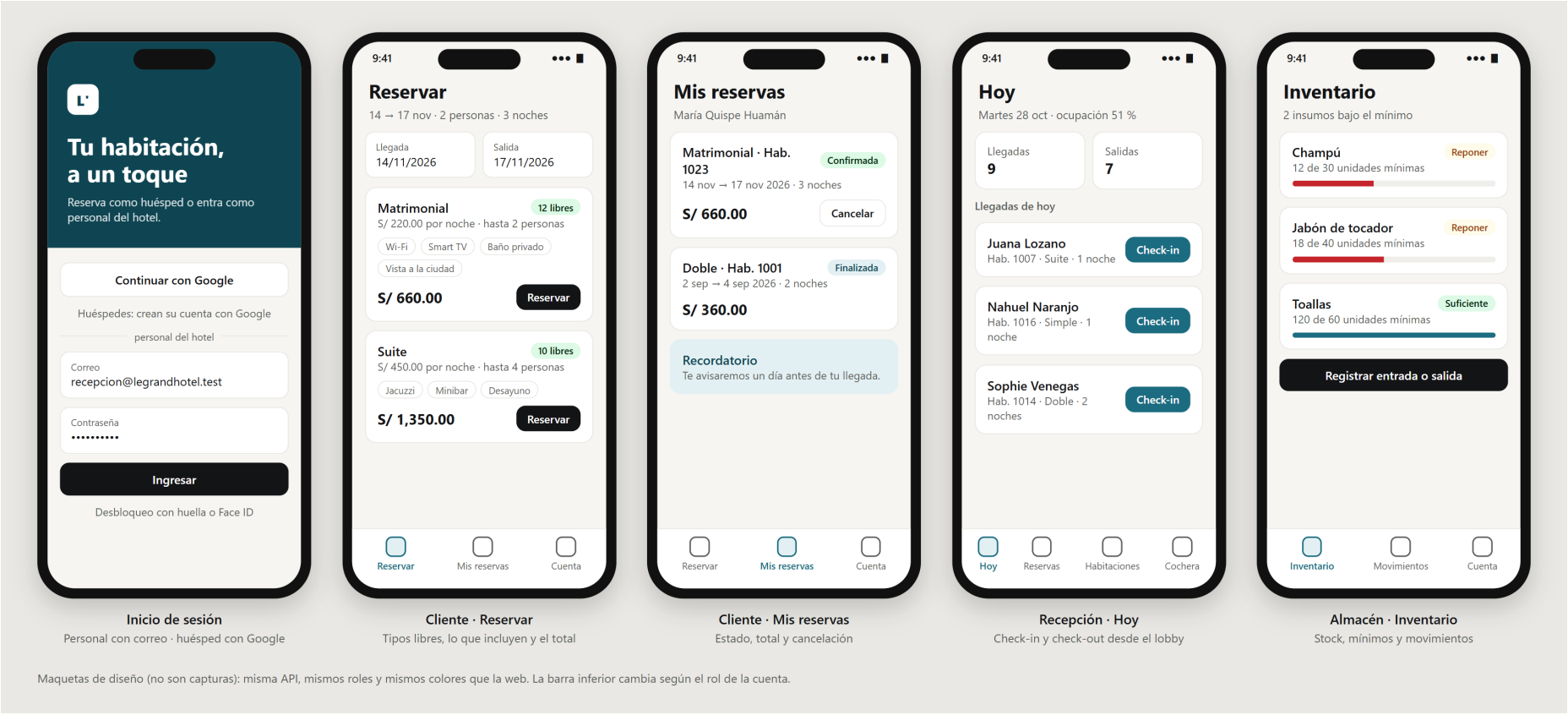


Figura 62. Maquetas de la app: inicio de sesión, Reservar y Mis reservas del cliente, «Hoy» de recepción e Inventario del almacén.

8.1 Tecnología

React Native con Expo y TypeScript. Con un solo código se generan las dos aplicaciones y se reutiliza lo que el equipo ya usa en la web: React, TypeScript, TanStack Query, Axios y los mismos tipos de datos.

Necesidad	En la app	Equivalente en la web
Navegación	Expo Router, con barra de pestañas según el rol	React Router
Llamadas a la API	TanStack Query y Axios con el mismo manejo de 401/403/429/5xx	Igual
Guardar la sesión	expo-secure-store: Keychain (iOS) y Keystore (Android)	sessionStorage
Inicio con Google	expo-auth-session con el enlace legrandhotel://auth/callback	Socialite con redirección
Notificaciones	expo-notifications (recordatorio de llegada, habitación lista, stock bajo)	—
Desbloqueo	Huella o Face ID (expo-local-authentication)	—
Pruebas	Jest y Testing Library; Maestro para recorridos completos	Vitest y Playwright

Opción evaluada	A favor	En contra	Decisión
React Native + Expo	Un código para Android e iOS; mismo lenguaje que la web; compila iOS sin Mac (EAS Build)	Funciones nativas muy específicas requieren módulos propios	Elegida
Flutter	Buen rendimiento y diseño uniforme	Otro lenguaje (Dart): no se reutiliza nada de la web	Descartada
Nativo (Kotlin + Swift)	Acceso total al sistema	Dos aplicaciones distintas: el doble de trabajo	Descartada

8.2 Seguridad y cambios en la API

- El token se guarda en el almacenamiento seguro del teléfono, nunca en texto plano, y se renueva con el mismo POST /auth/refresh de la web.
- La app no contiene secretos: las credenciales de Google y de la base de datos quedan solo en la API.
- La API casi no cambia: el login ya acepta un nombre de dispositivo (device_name) para que cada teléfono tenga su propio token. Faltará registrar el token de notificaciones de cada teléfono (POST /devices) y que el inicio con Google pueda volver a la app.

8.3 Plan de construcción

Etapa	Qué se construye
1	Proyecto Expo, login del personal y del cliente, sesión segura y barra de pestañas por rol
2	Cliente: completar registro, reservar y mis reservas
3	Recepción: «Hoy» con check-in y check-out, reservas y habitaciones; Almacén: inventario y movimientos
4	Notificaciones, desbloqueo con huella, pruebas y versión de prueba para Android e iOS

9. Conclusiones

1. El sistema cubre la operación completa del hotel. Con 77 endpoints REST y un formato único de respuesta, LeGrandHotel atiende reservas, estadías, habitaciones, huéspedes, cochera, inventario, personal, reportes y auditoría, y además la reserva en línea del huésped.

2. El acceso está separado y protegido. El personal entra con correo y contraseña y lo registran el Administrador o el Programador; los clientes entran solo con Google. Cada rol ve solo su menú, la API responde 403 aunque alguien escriba la dirección a mano y existe un solo Programador.

3. Las reglas del negocio las cuida el servidor. La API impide la doble reserva, dos huéspedes en la misma habitación, el stock negativo y la cochera ocupada (409), calcula los totales y deja cada cambio en la auditoría.

4. Las fallas se registran y se corrigen con evidencia. El registro de incidentes y el código de incidente que ve el usuario permitieron encontrar la causa de cada falla en el registro de la API y corregirla el mismo día, con una prueba automática nueva cuando correspondía.

5. La API es estable bajo carga y el siguiente paso está claro. En las pruebas k6 con clientes del portal no hubo ni un error con hasta 50 clientes a la vez y con 5 se cumplen todas las metas; la velocidad en la laptop la limita el servidor de desarrollo y se medirá de nuevo cuando se defina el servidor. La app Android e iOS reutilizará esta misma API.